

Architecture Evolution of Spring AI

EPL484 Software Evolution – Project 1 – Team X

⟨Member 1 (ID)⟩, ⟨Member 2 (ID)⟩

October 2026

- [1. Project choice and characteristics](#)
 - [1.1 What is Spring AI?](#)
 - [1.2 Eligibility \(Phase 1\)](#)
 - [1.3 Number and size of versions](#)
 - [1.4 Is the architecture documented?](#)
- [2. Methodology](#)
 - [2.1 Data collection](#)
 - [2.2 Data cleaning](#)
 - [2.3 Tools and libraries](#)
 - [2.4 Implementation steps](#)
 - [2.5 Choosing the number of clusters k](#)
 - [2.6 Quality metrics \(3.5\)](#)
 - [2.7 Additional analyses from the lectures](#)
- [3. Results and discussion](#)
 - [3.1 Size and its evolution](#)
 - [3.2 Class dependencies](#)
 - [3.3 Widely-used \(noise\) classes](#)
 - [3.4 Recovered architectures \(A1–A4\)](#)
 - [3.5 Quality of the architecture over time](#)
 - [3.6 Distance to the reference architecture \(MoJoFM\)](#)
 - [3.7 Omnipresent classes: JNode vs Bunch](#)
 - [3.8 Architectural smells and their evolution](#)
 - [3.9 Lehman's laws: dependency changes and work rate](#)
- [4. Architecture recovery with AI \(latest version, 2.1.0-M1\)](#)
- [5. Reflection](#)
 - [5.1 Comparing the architectures of the latest version](#)
 - [5.2 Does the AI tool help? How do information and prompts influence the result?](#)
 - [5.3 Which method for other systems?](#)
- [6. Threats to validity](#)
- [References](#)
- [Appendix A – Full metric tables](#)

Code and data: GitHub repository `ep1484.fall26.project1.teamX` (pipeline in `scripts/`, data in `data/`, all computed results in `results/`). Every number in the tables of this report is generated from the result files by `scripts/report_tables.py`.

1. Project choice and characteristics

1.1 What is Spring AI?

[Spring AI](#) is the Spring ecosystem's application framework for AI engineering. Its goal is to bring Spring's principles (portability, modularity, POJOs, dependency injection) to applications that use generative AI models. It offers:

- **Portable model APIs** for chat, embedding, image generation, audio (transcription, text-to-speech) and moderation (ChatModel, EmbeddingModel, ImageModel, ...), with synchronous and streaming calls;
- **adapters to ~12 AI providers** (OpenAI, Anthropic, Amazon Bedrock, Google GenAI/Vertex AI, Ollama, Mistral AI, DeepSeek, ElevenLabs, Stability AI, ...);
- the fluent **ChatClient** API with an **advisor** (interceptor) chain for chat memory, RAG, logging and guard-rails;
- **structured output** (LLM text → POJOs), **tool/function calling** and **MCP** (Model Context Protocol) client/server support;
- **Retrieval-Augmented Generation**: an ETL pipeline (document readers, splitters), a portable **VectorStore** API with a SQL-like metadata filter language and ~20 vector-database adapters;
- **observability** (Micrometer) and **Spring Boot auto-configuration/starters** for everything above.

The project lives in the spring-projects GitHub organisation (Apache 2.0). Its first commit was made on 2023-07-24. It has 4,249 commits, 4,007 of them after 1/1/2024, and it is very actively developed.

1.2 Eligibility (Phase 1)

Criterion	Spring AI
≥ 10 major/minor versions	48 tags: 4 GA (0.8.0, 1.0.0, 1.1.0, 2.0.0), 21 feature milestones, 4 RCs, 19 patches. Strictly 5 major.minor lines (0.8, 1.0, 1.1, 2.0, 2.1); 25 feature releases if milestones are counted (see note)
Repository on GitHub	spring-projects/spring-ai
Latest version ≥ 10K LOC	2.1.0-M1: 1,341 production .java files, 95,546 non-comment, non-blank LOC
Not archived / not a fork	✓ / ✓ (original repository)
≥ 50 commits since 1/1/2024	4,007
Created before 2024	2023-07-24

Note on versions. Spring projects deliver all features of a line through public **milestones** (M1...M8), followed by a release candidate and a GA. For example, the 1.0 line grew from 489 to 828 classes during its eight milestones. We therefore treat milestones as feature releases (agreed with the instructor in the approval e-mail <to be confirmed>).

1.3 Number and size of versions

We analyse the **continuous sequence of all 25 feature releases** from 0.8.0 (Feb 2024) to 2.1.0-M1 (Sep 2026), i.e. every milestone and GA with no gaps. Table 1 gives the size of each version. *Modules* are the Maven modules that contain code. *Class files* include inner and anonymous classes. *Top-level classes* are compilation units. *Dependencies* are distinct class-to-class dependencies between top-level classes.

Table 1 – Size of the analysed versions.

Version	Date	Modules	Packages	Class files	Top-level classes	Δ classes	Dependencies	Deps/class
0.8.0	2024-02-26	21	84	543	290	–	827	2.89
1.0.0-M1	2024-05-28	35	143	957	489	+68.6%	1593	3.30
1.0.0-M2	2024-08-23	42	186	1371	667	+36.4%	2329	3.54
1.0.0-M3	2024-10-20	45	200	1438	707	+6.0%	2680	3.83
1.0.0-M4	2024-11-20	47	210	1496	739	+4.5%	2895	3.95
1.0.0-M5	2024-12-23	48	229	1576	760	+2.8%	3053	4.05
1.0.0-M6	2025-02-14	51	231	1542	775	+2.0%	3104	4.05
1.0.0-M7	2025-04-10	104	240	1619	828	+6.8%	3241	3.98
1.0.0-M8	2025-04-30	104	239	1572	812	-1.9%	3097	3.88
1.0.0	2025-05-19	102	228	1562	783	-3.6%	2969	3.87
1.1.0-M1	2025-09-08	110	252	1717	847	+8.2%	3194	3.86
1.1.0-M2	2025-09-22	113	265	1807	895	+5.7%	3365	3.84
1.1.0-M3	2025-10-03	115	269	1856	911	+1.8%	3387	3.78
1.1.0-M4	2025-11-01	116	270	1903	928	+1.9%	3472	3.80
1.1.0	2025-11-12	117	270	1908	929	+0.1%	3440	3.76
2.0.0-M1	2025-12-11	120	275	1969	962	+3.6%	3590	3.79
2.0.0-M2	2026-01-23	130	286	2025	995	+3.4%	3746	3.82
2.0.0-M3	2026-03-16	130	307	2177	1112	+11.8%	4181	3.81
2.0.0-M4	2026-03-26	130	307	2184	1113	+0.1%	4181	3.80
2.0.0-M5	2026-04-27	121	282	1899	1007	-9.5%	3583	3.60
2.0.0-M6	2026-05-08	118	279	1915	997	-1.0%	3554	3.61
2.0.0-M7	2026-05-22	112	273	1894	980	-1.7%	3520	3.64
2.0.0-M8	2026-05-27	112	274	1902	983	+0.3%	3538	3.64
2.0.0	2026-06-12	112	278	1901	988	+0.5%	3451	3.54
2.1.0-M1	2026-09-24	116	286	2015	1039	+5.2%	3725	3.63

1.4 Is the architecture documented?

Partly. The [reference documentation](#) describes the concepts (models, prompts, embeddings, tools, RAG, advisors, vector stores, MCP) and contains a high-level integration diagram (“connect your *Data* and *APIs* with the *AI Models*”). The repository has a `design/` folder with short design records. *Design 02 – Spring Boot Modularity* is the only document that states an architectural rule: the core must not depend on Spring Boot, auto-configuration modules depend on the core, and starters contain no code. The Maven build enforces this rule (maven-enforcer-plugin). There is **no documented component/connector view** and **no documented decomposition**. The module structure and the package naming (`*.autoconfigure`, `vectorstore.<vendor>`, `<provider>.api`) are the only other architectural evidence.

2. Methodology

Figure 1 summarises the pipeline. All steps are scripts in the repository and can be re-run with the commands in `README.md`.

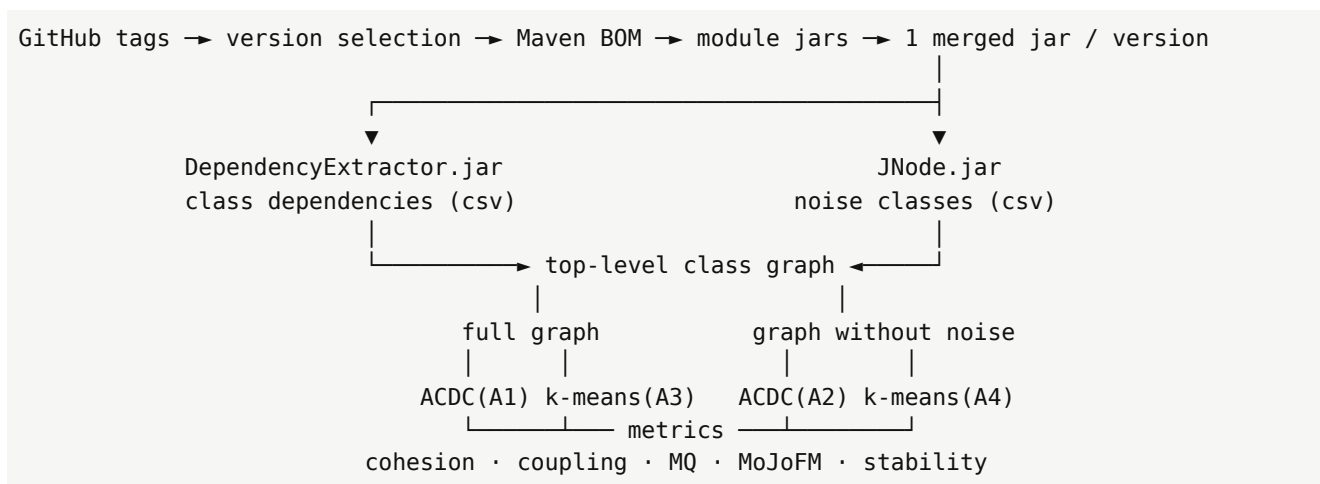


Figure 1 – Analysis pipeline.

2.1 Data collection

1. **Releases.** All 48 tags were listed with `git ls-remote --tags` (`data/tags.csv`). Creation dates and the Phase 1 statistics come from a blob-less clone (`scripts/project_stats.sh`).
2. **Binaries.** Spring AI is a multi-module Maven project (21 modules in 0.8.0, 116 in 2.1.0-M1), and its module structure changes often (e.g. the 1.0.0-M7 split of `spring-ai-core`). To analyse the *whole system* consistently, `scripts/collect_versions.py` reads each release’s **BOM** (`spring-ai-bom`, the official list of the release’s artifacts). It downloads every listed module jar from Maven Central or, for old milestones, from `repo.spring.io/milestone`. It then merges all `org/springframework/ai/**.class` files into **one jar per version** (`data/workspace/<v>/bin/`). This is the input format of the course tools.

2.2 Data cleaning

- **Excluded releases.** 19 *patch* releases (bug fixes on maintenance branches, published in parallel with newer lines, e.g. 1.0.9 and 2.0.0 on the same day) and 4 *release candidates* (feature-frozen, practically identical to the following GA). This left **25 of 48** releases.
- **Excluded artifacts.** *Starters* (no code), *spring-ai-test* (test utilities), the BOM and docs. module-info/package-info classes and third-party classes were also dropped. Classes that appear in two modules (split packages) were kept once.
- **Inner classes.** DependencyExtractor and JNode work on class files. For architecture recovery we fold inner/anonymous classes (A\$B, A\$1) into their top-level class and drop the resulting self-dependencies. A top-level class is noise if any of its class files is flagged by JNode.
- **Isolated classes.** 2-15 classes per version have no dependency to or from another Spring AI class. They cannot be clustered from dependencies and are left out of A1-A4 (column *Isolated* in Table 3).

2.3 Tools and libraries

Step	Tool / library
Class dependencies (3.2)	Course DependencyExtractor.jar (bytecode; EXTENDS, IMPLEMENTS, METHOD_CALL, METHOD_PARAMETER, FIELD_REFERENCE, TYPE_USAGE, ...)
Noise classes (3.3)	Course JNode.jar , with two documented fixes (below)
ACDC (A1, A2)	ACDC (Tzerpos & Holt) – Java port from USC ARCADE (tools/acdc/, BSD licence). The York wiki link was not reachable from our environment
Clustering (A3, A4)	scikit-learn KMeans, TruncatedSVD; kneed (elbow)
Metrics, tables, plots	Python 3.13, numpy, scipy, matplotlib
PDF	pandoc + headless Chromium

Fixes to JNode. JNode needed more than 2 hours per 2,000-class version and broke down on the largest versions. We decompiled it (CFR) and recompiled three methods; tools/jnode-patch/ documents the patch:

1. *Performance* – a class-name lookup that scanned the whole class list on every step of the subgraph extraction, and a set-membership test written as an iteration, were replaced by hash lookups. The output is **byte-identical** to the original on 0.8.0, 1.0.0-M1 and 1.0.0-M2. The run time dropped from ~9 min to ~1.3 min for 1.0.0-M1.
2. *Precision* – JNode’s CR model rounds weights to 3 decimals and initialises every weight to $\text{round}(1/n, 3)$. For **$n > 2,000$ class files** this rounds to **0**. The shortest-path weights then become 0, $1/0 = \infty$, every SIG is normalised to 0, and no noise is found. This happened for exactly the four versions with more than 2,000 class files (2.0.0-M2/M3/M4, 2.1.0-M1). We re-ran all versions with 4-decimal rounding. On the 20 versions where the original worked, the noise sets agree with Jaccard 0.88–1.00 (Table 3).
3. *Threshold fallback* (in our scripts, not in JNode) – JNode flags a class if its normalised $\text{SIG} \geq \text{mean} + 1\sigma$. For 0.8.0 this limit (1.004) is above the maximum (1.0), so nothing is

flagged. Only for that case we clamp the limit to the maximum SIG, i.e. the classes with the highest SIG are noise.

Changes to ACDC. (a) Optional parameters `maxClusterSize` (default 20) and pattern set (b BodyHeader, s SubGraph, o OrphanAdoption), so that we can run the experiment suggested in the tutorial. (b) A hash map for node lookup when reading the RSF file (same output, linear instead of quadratic time). (c) **Subsystem naming.** ACDC was written for C file names and names each subsystem after the *base name* of its dominator (the text before the last dot, `foo.c` $\rightarrow$ `foo`) + `.ss`. For a Java class name this base name is the *package*, so all subsystems whose dominators share a package received the same name and were merged when the output was read. We therefore pass file-style names (`pkg.Class.java`) to ACDC and strip the suffix from the output; each subsystem is then named after its dominator class (e.g. `openai.OpenAiChatModel.ss`). Input: `data/rsf/<v>_full.rs` / `_nonoise.rs` with one `depends A B` line per distinct dependency.

2.4 Implementation steps

1. `collect_versions.py` $\rightarrow$ merged jars, `data/versions.csv`, `data/modules/`.
2. `run_tools.sh` / `run_jnode_pool.sh` $\rightarrow$ `data/dependencies/<v>.csv`, `data/noise/<v>.csv`.
3. `recover.py`, for each version: build the top-level class graph; write the two RSF files; **A1/A2** ACDC on the full graph and on the graph without noise classes (noise nodes and all their edges removed); **A3/A4** k-means on the same two graphs; **PKG** – the developers’ Java package decomposition as a reference; evaluate all of them with `metrics.py`.
4. `acdc_params.py` (ACDC experiment), `ai_inputs.py` / `ai_eval.py` (Phase 4), `noise_fanin.py`, `compare_noise.py`, `module_changes.py`, `plots.py`, `report_tables.py` + `build_report.sh` (this report).

Feature vectors for k-means. k-means needs points in a vector space, so each class is represented by its *dependency profile*: its row of the symmetric adjacency matrix $A + A^T + I$ (the classes it uses, the classes that use it, and itself). Rows are L2-normalised and reduced to 64 dimensions with truncated SVD (LSA). On the normalised vectors, Euclidean k-means approximates clustering by cosine similarity of dependency profiles. Classes with similar neighbourhoods end up together. We used `n_init = 10` and `random_state = 42`.

2.5 Choosing the number of clusters k

For every version and both graphs we ran k-means for $k = 4 \dots n/3$ and recorded the **inertia** (within-cluster sum of squares) and the **silhouette** coefficient (cosine). k was chosen with the **elbow method**, located automatically with the *Kneedle* algorithm. Figure 2 shows both curves for 2.1.0-M1. The silhouette rises steeply up to the elbow and then flattens. Its maximum lies at clusters of only 4–11 classes on average (e.g. $k = 76$ for the 284 classes of 0.8.0), which is too fine to be an architecture: such a “component” is little more than a class with its helpers. The elbow gives clusters of 6–12 classes on average, slightly coarser than ACDC (5–6 classes per cluster). At the elbow the silhouette has already reached 91–98 % of its maximum (77–79 % only for the small 0.8.0). The resulting k grows from 36 (0.8.0) to 84 (A3) / 76 (A4) in 2.1.0-M1 (Table 5).

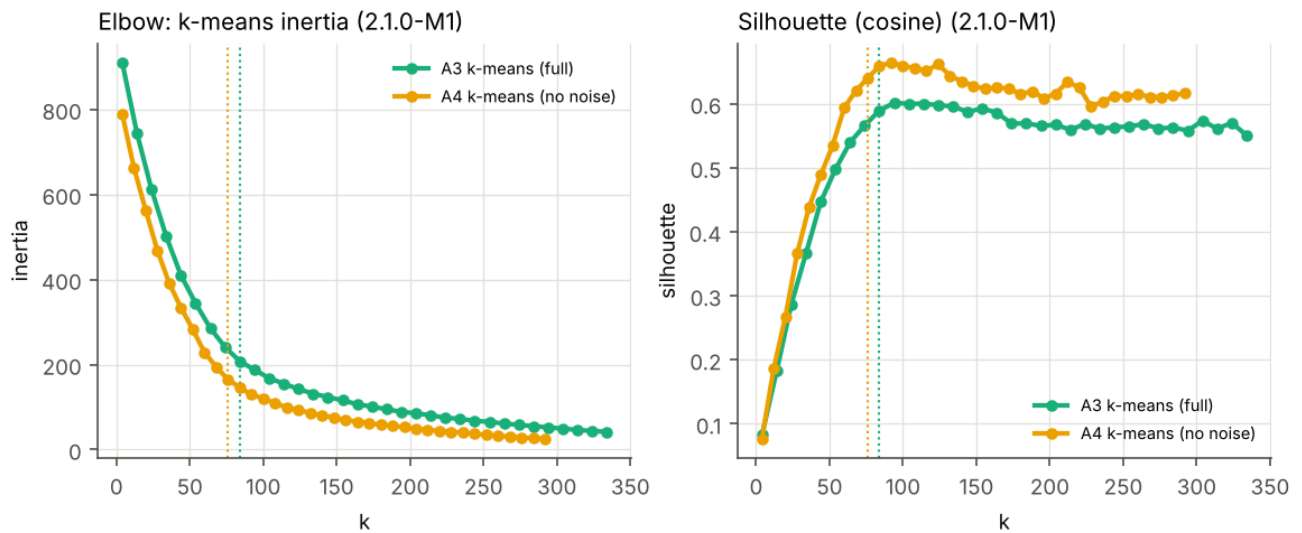


Figure 2 – k selection for 2.1.0-M1: elbow on the inertia (left) and silhouette (right); dotted lines = chosen k.

2.6 Quality metrics (3.5)

For a clustering into k clusters, with μ_i the dependencies inside cluster i (N_i classes) and ε_{ij} the dependencies between clusters i and j (directed, unweighted class graph), we use the definitions of the course (Lecture 6-7, “Ποια θεωρείται καλή διάτμηση”):

- **Cohesion** (intra-connectivity) $A_i = \mu_i / N_i^2$. The lecture allows μ_i / N_i^2 or $\mu_i / (N_i(N_i - 1))$. We use N_i^2 because it is defined for one-class clusters (packages and ACDC produce some). We report the mean over clusters.
- **Coupling** (inter-connectivity) $E_{ij} = \varepsilon_{ij} / (2 \cdot N_i \cdot N_j)$, averaged over all cluster pairs.
- **MQ - Modularization Quality** (course definition, Mancoridis et al. 1998): $MQ = (1/k) \cdot \sum A_i - (2/(k(k-1))) \cdot \sum E_{ij}$, in $[-1, 1]$. This is the **main quality measure** of the assignment. Because A_i divides by N_i^2 , MQ favours many small, dense clusters, so it is only comparable between architectures with a similar number of clusters.
- Supplementary measures, independent of k : **TurboMQ/k**, the mean cluster factor $2\mu_i / (2\mu_i + \varepsilon_i)$ of the later Bunch MQ (Mitchell & Mancoridis 2006), where ε_i counts all dependencies crossing the border of cluster i ; and **intra deps**, the share of all dependencies that stay inside a cluster. TurboMQ (the sum, which grows with k) is given in the appendix.

2.7 Additional analyses from the lectures

- **MoJoFM** (Lecture 6-7): $MoJoFM(A, R) = (1 - mno(A, R) / \max mno(\cdot, R)) \cdot 100 \%$, where mno is the minimum number of *Move* and *Join* operations that turn architecture A into reference R (100 % = identical). It is computed with the original MoJo 2.0 implementation (tools/mojo.jar). The system has no expert (“ground-truth”) architecture, so we use the developers’ **package structure** as the reference for every version and, for the latest version, also the **AI architecture** of Phase 4. MoJoFM between the architectures of consecutive versions measures **architectural stability**; we also report the adjusted Rand index (ARI).
- **Omnipresent classes with the Bunch rule** (Lecture 6-7): in-degree $> 3 \times$ average in-degree. We compare it with JNode (Constantinou et al. 2015), and, as in the lecture’s

comparison of “System”, “Noise” and “Bunch”, we measure how close ACDC gets to the reference architecture without each set.

- **Architectural smells and their evolution** (Lecture 5; Fontana et al. 2016, Sas et al. 2019), each normalised by the number of classes or packages: *cyclic dependency* (packages/classes in a dependency cycle, i.e. a strongly connected component of size > 1 , found by depth-first search); *hub-like dependency* (classes with fan-in and fan-out above the system medians and $|\text{fan-in} - \text{fan-out}| < 0.25 \cdot (\text{fan-in} + \text{fan-out})$); *unstable dependency* (packages, instability $I = C_e / (C_a + C_e)$, of which more than 30 % of the package dependencies point to less stable packages - the Arcan threshold); *god component* (packages with more classes than mean $+ 2\sigma$ of the package sizes, since the lecture leaves the threshold open).
- **Lehman’s laws** (Lecture 1-2): besides size and dependency growth, we count the **dependencies added and removed** per version (as in the lecture’s Eclipse example for law II) and the **commits per month** between the analysed releases (law IV, conservation of organisational stability).

3. Results and discussion

3.1 Size and its evolution

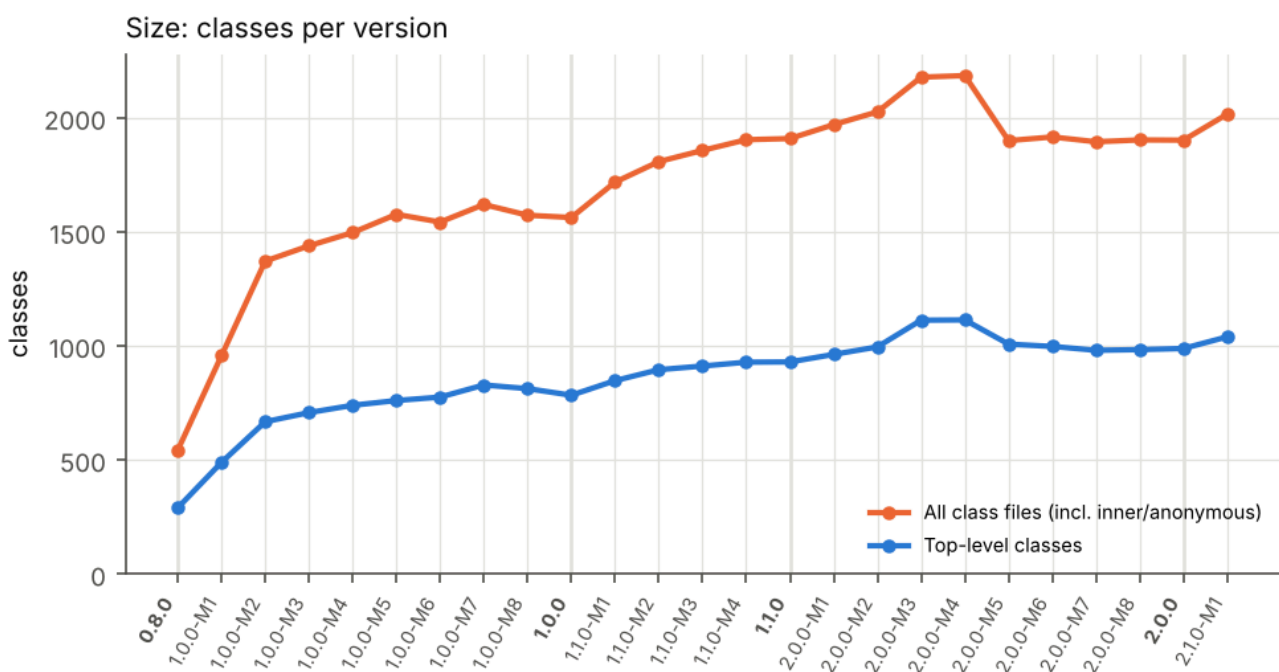


Figure 3 – Number of classes per version (GA releases in bold).

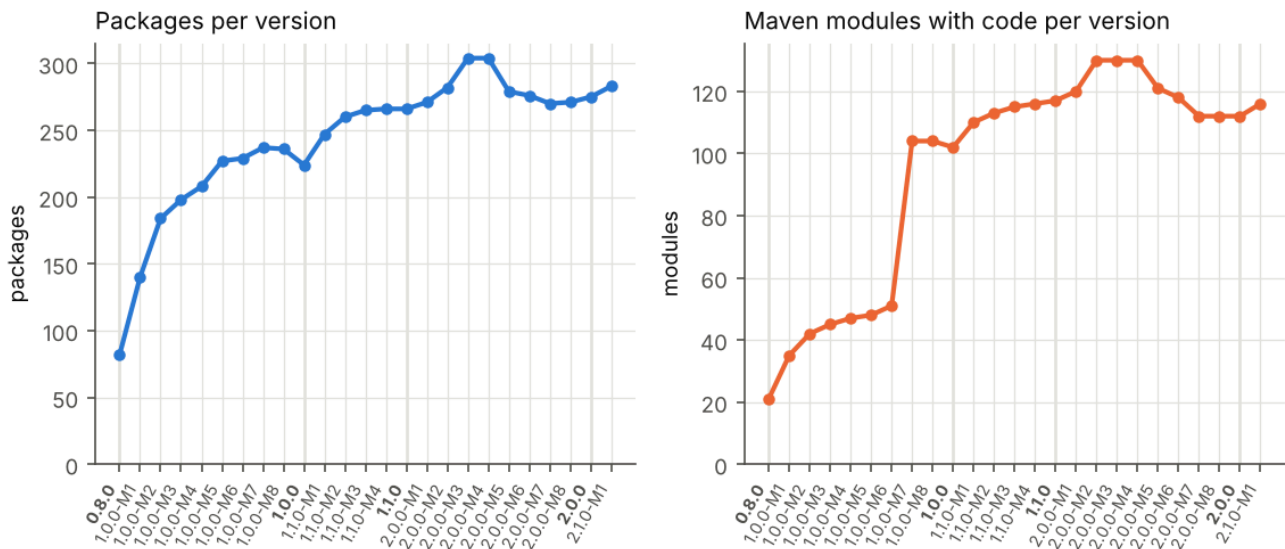


Figure 4 – Packages and Maven modules per version.

The system grew from **290 to 1,039 top-level classes (×3.6)** and from 543 to 2,015 class files in 31 months (Table 1, Figures 3–4). The growth is **not uniform**:

- **0.8.0 → 1.0.0-M2 (×2.3 in 6 months)**: the young framework adds model providers (Anthropic, Mistral, ZhiPu, MiniMax, Vertex Gemini, Watsonx, ...), vector stores (Cassandra, Elasticsearch, Qdrant, Redis, ...) and the ChatClient/advisor API (+198 classes in spring-ai-core in M2 alone).
- **1.0.0-M3 → 1.1.0 (slower, −4 % to +8 % per release)**: consolidation before and after the first GA. **1.0.0-M8 → 1.0.0 even shrinks (−3.6 %)**: the RAG and chat-memory APIs were reworked and the Watsonx adapter removed for the GA.
- **2.0.0-M3 (+11.8 %)**: the MCP annotation framework (+190 classes) is moved into Spring AI, and the Anthropic adapter is rewritten on the official SDK.
- **2.0.0-M5 (−9.5 %)**: clean-up for the 2.0 GA. The OpenAI adapter is rewritten (−92 classes), and Azure OpenAI, Vertex AI Gemini and ZhiPu AI are removed (moved to the community organisation).

The **number of modules** shows a structural event the class count does not. In **1.0.0-M7** the monolithic spring-ai-core (542 class files) and spring-ai-spring-boot-autoconfigure (185) were split into spring-ai-model, -client-chat, -vector-store, -commons, -rag and **one auto-configuration module per provider/store**. The module count jumped from 51 to 104 (+57/−4 modules, Table 9), but the class count changed by only +7 %. It was a **re-modularisation**, not a growth step.

Lehman’s laws. *Continuing change (I)* and *continuing growth (VI)* are clearly visible: 25 releases in 31 months, and every release adds or removes modules (Table 9). Growth is fast in the first months and then slows down to a few percent per release. This is the “inverse-square” shape that Turski’s model associates with the growing effort needed to understand an ever-larger system, and it is consistent with *conservation of familiarity (V)*: after 1.0 no release changes more than ~12 % of the system. The size curve also shows *self-regulation (III)*: phases of expansion (M1-M2, 1.1-M1-M2, 2.0-M2-M3) are followed by releases that shrink or restructure the system (1.0.0, 2.0.0-M5...M7).

3.2 Class dependencies

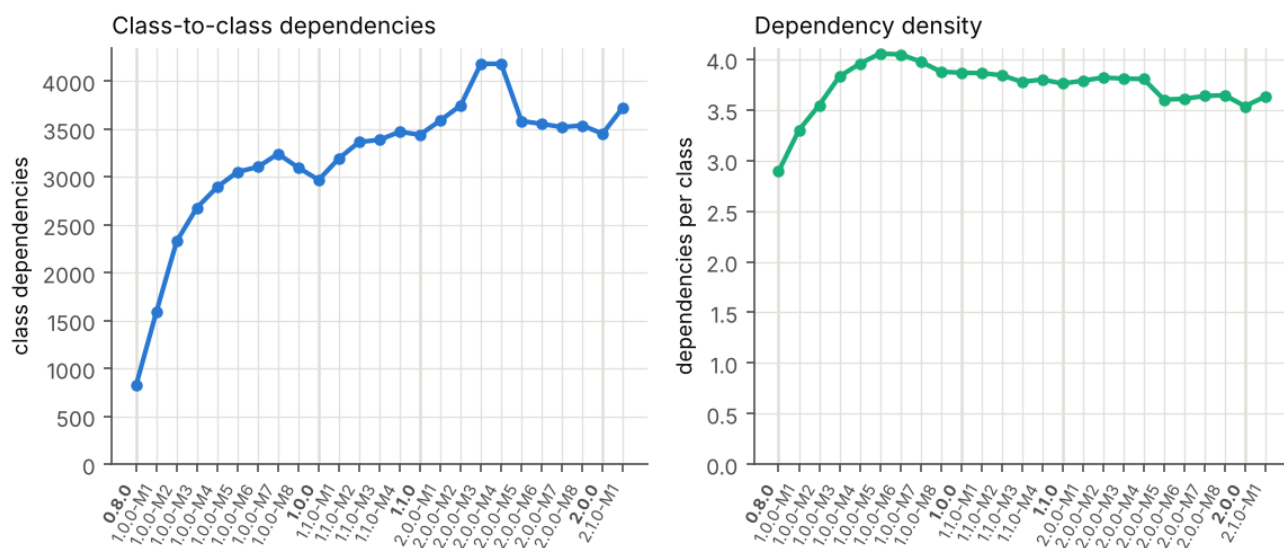


Figure 5 – Class-to-class dependencies (left) and dependencies per class (right).

Dependencies grew from **827 to 3,725 ($\times 4.5$)**, faster than the classes ($\times 3.6$). Density rose from 2.9 to **4.1 dependencies per class at 1.0.0-M6**, the version just before the module split (Figure 5). From 1.0.0-M7 onwards it **falls** to 3.6–4.0 and stays there, even though the system keeps growing. This is *increasing complexity (II)* together with the counter-measure the law predicts: complexity grows “unless work is done to maintain or reduce it”. The 1.0.0-M7 re-modularisation and the 2.0 clean-up are exactly such *anti-regressive* work.

3.3 Widely-used (noise) classes

Table 3 – Noise classes (JNode) per version. *JNode-3/4 noise files* = class files flagged by the original (3-decimal) and fixed (4-decimal) JNode; *Top-30 fan-in flagged* = how many of the 30 most-used classes are noise.

Version	Connected classes	Isolated	Noise (top-level)	Noise %	JNode-3 noise files	JNode-4 noise files	Jaccard 3 vs 4	Top-30 fan-in flagged	Fallback
0.8.0	284	2	21	7.39%	0	0	1.0	13	yes
1.0.0-M1	479	4	95	19.83%	201	199	0.99	29	
1.0.0-M2	650	7	127	19.54%	295	286	0.9695	30	
1.0.0-M3	691	9	114	16.5%	242	214	0.8843	30	
1.0.0-M4	723	9	114	15.77%	218	196	0.8991	30	
1.0.0-M5	744	9	113	15.19%	227	201	0.8855	30	
1.0.0-M6	756	11	147	19.44%	295	277	0.939	29	
1.0.0-M7	804	11	72	8.96%	129	130	0.9923	22	
1.0.0-M8	787	12	77	9.78%	139	141	0.9858	20	
1.0.0	755	13	71	9.4%	133	133	1.0	22	
1.1.0-M1	814	13	91	11.18%	161	161	1.0	21	
1.1.0-M2	863	13	108	12.51%	181	184	0.9837	24	
1.1.0-M3	884	13	106	11.99%	183	183	1.0	24	
1.1.0-M4	901	13	108	11.99%	198	198	1.0	24	
1.1.0	901	14	111	12.32%	199	200	0.995	24	
2.0.0-M1	933	15	112	12.0%	202	209	0.9665	24	
2.0.0-M2	968	13	112	11.57%	0	209	0.0	24	
2.0.0-M3	1087	11	123	11.32%	0	188	0.0	22	
2.0.0-M4	1088	11	124	11.4%	0	189	0.0	22	
2.0.0-M5	986	9	125	12.68%	192	193	0.9948	20	
2.0.0-M6	976	9	123	12.6%	193	193	1.0	21	
2.0.0-M7	960	8	123	12.81%	190	190	1.0	21	
2.0.0-M8	963	8	126	13.08%	194	194	1.0	21	
2.0.0	966	10	112	11.59%	180	180	1.0	15	
2.1.0-M1	1016	10	116	11.42%	0	188	0.0	16	



Figure 6 - Noise classes per version (absolute and % of connected classes).

JNode flags **21-147 top-level classes (7-20 %)** per version. The noise classes have a fan-in 3-9 times higher than the other classes (13.1 vs 2.4 on average, results/noise_vs_fanin.csv). In 0.8.0-1.0.0-M6 they are exactly the classes one would call omnipresent: Document,

ModelRequest, ModelResponse, ChatOptions, ModelOptionsUtils, Message, Filter, EmbeddingModel, FunctionCallback (29–30 of the 30 classes with the highest fan-in).

Two observations need care:

1. **The share drops abruptly from 19 % to 9 % at 1.0.0-M7.** The core API classes are still widely used, but the module split changed the shape of the graph. JNode computes SIG only *inside the subgraph reachable from each class* and then normalises globally with mean + 1σ . After the split, the per-module auto-configuration classes reach long chains of classes; this presumably raises the mean SIG, so fewer classes exceed the threshold.
2. **In 2.x JNode drifts away from fan-in.** In 2.1.0-M1 only 16 of the 30 most-used classes are flagged. Document (fan-in 71), Filter (61) and EmbeddingModel (47) are *not* flagged, while MCP transports (56 of the 116 noise classes are in `mcp.*`) and Anthropic adapter classes are. JNode’s notion of “noise” is a *reachability/significance* measure, not a usage count. It depends on how the classes are connected, and the 2.x MCP framework forms a large, densely connected subgraph. Removing these classes still helps the clustering (3.4), but the noise set of 2.x versions should not be read as “the utility classes of Spring AI”.

3.4 Recovered architectures (A1-A4)

Table 4 – Mean over the 25 versions.

Architecture	k	Largest cluster	Cohesion	Coupling	MQ (course)	TurboMQ	TurboMQ/k	Intra deps	ARI vs packages
A1 ACDC (full)	149	48	0.2329	0.00238	0.2305	61.1	0.412	0.380	0.244
A2 ACDC (no noise)	144	26	0.2338	0.00157	0.2322	76.0	0.535	0.525	0.303
A3 k-means (full)	80	27	0.1371	0.00215	0.1349	41.9	0.528	0.332	0.268
A4 k-means (no noise)	72	23	0.1556	0.00108	0.1545	50.6	0.704	0.573	0.302
Packages (full)	242	22	0.1423	0.00312	0.1391	48.0	0.201	0.210	1.000
Packages (no noise)	210	18	0.1484	0.00265	0.1457	55.6	0.271	0.303	1.000

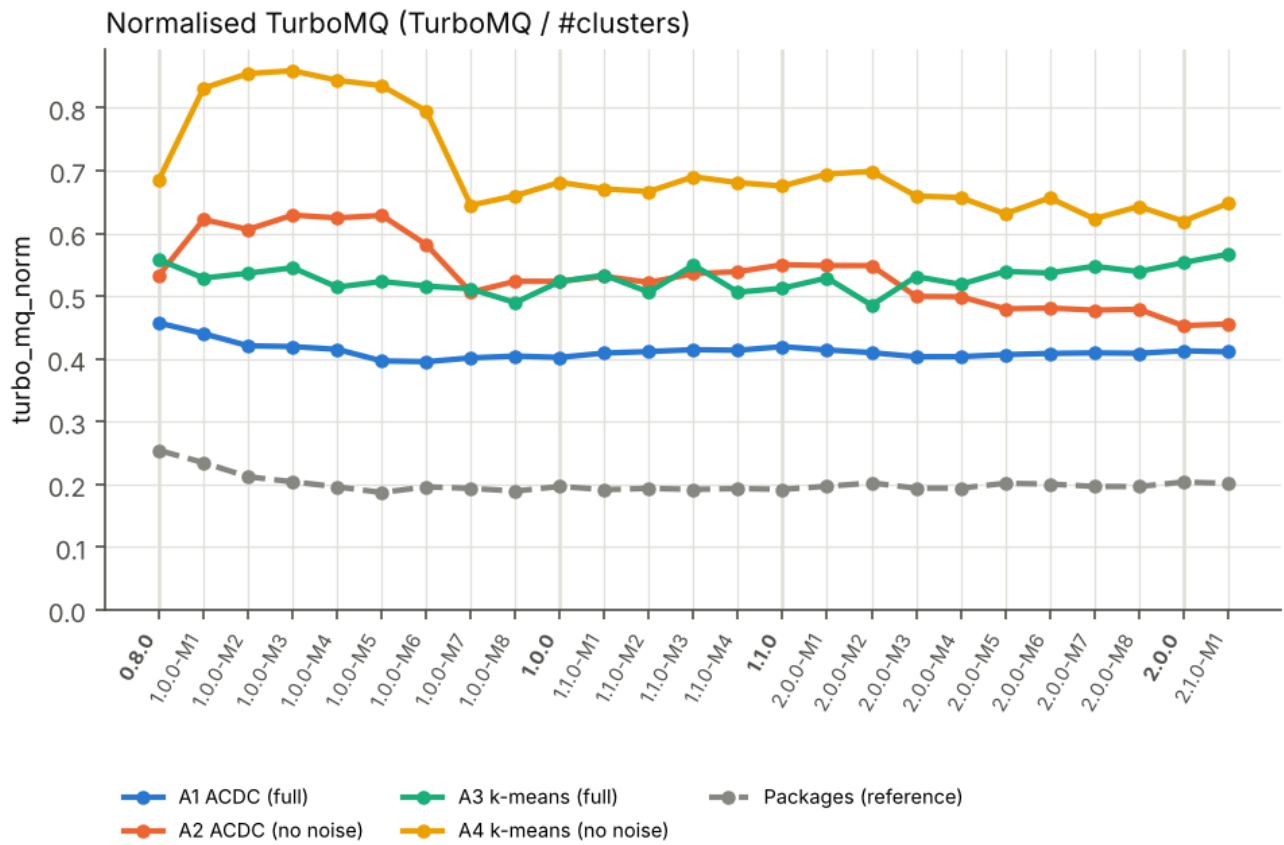


Figure 7 - Normalised TurboMQ (TurboMQ/k) per version and architecture.

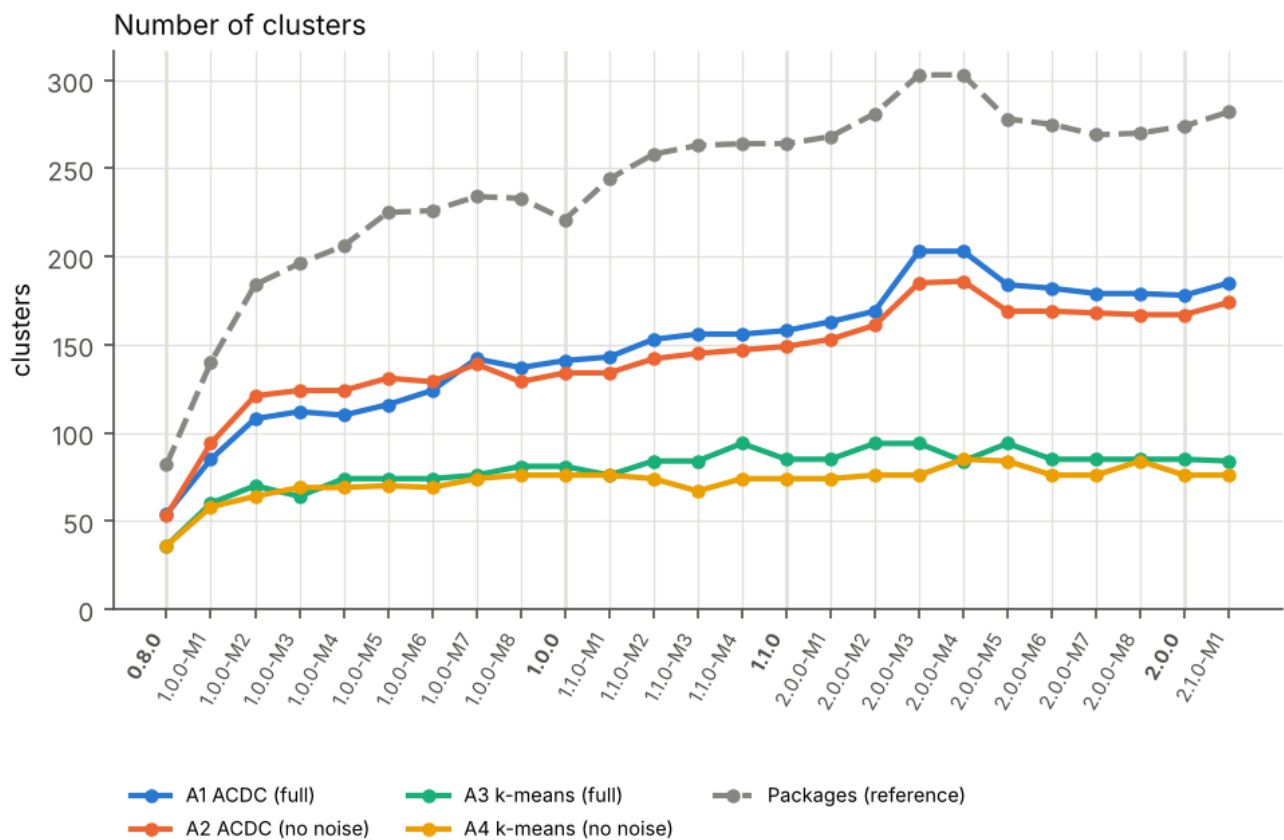


Figure 8 - Number of clusters per version.

Size of the recovered architectures. ACDC produces 54 (0.8.0) to 185 (2.1.0-M1) clusters, k-means 36 to 84 (A3) / 76 (A4). ACDC's clusters are less balanced: its largest cluster has 34-

76 classes (50 in 2.1.0-M1), whereas k-means clusters never exceed 35 classes. ACDC leaves almost no singletons (1.7 per version on average; OrphanAdoption assigns orphans to the subsystem they depend on most). The package decomposition has ~240 packages, 54 of them with a single class (Table 4).

Quality by the course MQ. ACDC has by far the highest MQ in every version (0.22–0.24; mean 0.231 for A1 and 0.232 for A2). k-means (0.135 / 0.155) and the package structure (0.139) score much lower. MQ is dominated by its cohesion term (0.23 for ACDC vs 0.14–0.16), while all coupling terms are ≈ 0.002 . ACDC’s many small, dense subgraph clusters (5–6 classes on average) are exactly what MQ rewards. By MQ, k-means is not better than the packages.

Quality by the k-independent measures. On TurboMQ/k and on the share of internal dependencies, all algorithmic architectures are **far more modular than the package structure**. On average 38 % (A1) and 33 % (A3) of the dependencies stay inside a cluster, against 21 % inside a package, and TurboMQ/k is 0.41–0.70 vs 0.20. Spring AI’s packages are organised by *feature* (chat.prompt, chat.messages, chat.model, ...) and by *vendor* (openai, openai.api, ...), and features are used across packages. The recovery algorithms group classes by *collaboration*.

- **ACDC vs k-means.** ACDC has the higher cohesion and MQ (0.23 vs 0.13–0.15) because its many small subgraph clusters are dense. k-means has the higher TurboMQ/k (0.53 vs 0.41 on the full graph) and fewer, more balanced clusters. ACDC keeps slightly more dependencies inside clusters on the full graph (38 % vs 33 %); without noise k-means is ahead (57 % vs 53 %).
- **Effect of noise removal (A1→A2, A3→A4).** Removing the noise classes raises TurboMQ/k by **+0.12 (ACDC) and +0.18 (k-means)**, the intra-cluster share from 38 → 53 % and 33 → 57 %, and lowers coupling by a third (ACDC) to a half (k-means). The course MQ hardly changes (A1 0.231 → A2 0.232; A3 0.135 → A4 0.155), because removing classes does not change how dense the small clusters are. The benefit shows in the k-independent measures and in the distance to the package structure (MoJoFM, 3.6). It also makes the clusterings more stable over time (3.5). Hub classes such as Document or ChatOptions connect almost every cluster to every other one; without them the true subsystems separate. The effect is strongest for 1.0.0-M1...M6 (TurboMQ/k up to 0.63 for A2 and 0.86 for A4), where JNode removed ~19 % of the classes, and smaller after 1.0.0-M7 (fewer noise classes, see 3.3).

ACDC parameters (tutorial suggestion). Table 6 shows the experiment on 2.1.0-M1. (i) **BodyHeader has no effect:** bso and so give identical results. (ii) **OrphanAdoption is essential:** without it, ACDC’s last step (ClusterLast) puts all unclustered classes into one cluster of 406 classes, and TurboMQ/k drops from 0.41 to 0.34. (iii) The **maximum cluster size** matters only below 20: 5 gives 213 smaller clusters with lower TurboMQ/k (0.39), 10 gives 192 (0.40), and 20, 40 or 80 give practically the same result (184–185 clusters, 0.411–0.412). We therefore kept the default (20, bso).

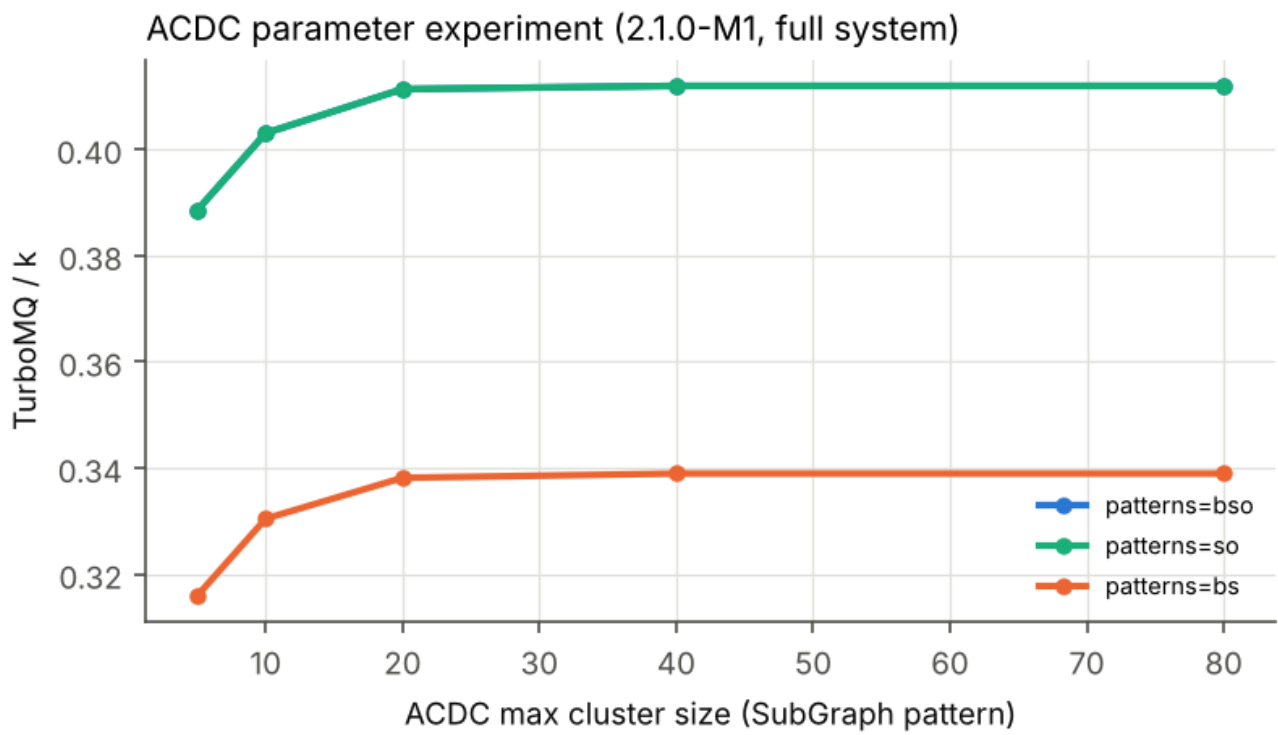


Figure 9 - ACDC parameter experiment (2.1.0-M1, full graph); the bso line lies exactly under so.

Table 6 - ACDC parameter experiment (2.1.0-M1).

Graph	Patterns	Max size	k	Largest	TurboMQ/k	Intra deps
full	bso	5	213	57	0.3885	0.3742
full	bso	10	192	57	0.403	0.3863
full	bso	20	185	50	0.4113	0.404
full	bso	40	184	50	0.4119	0.4067
full	bso	80	184	50	0.4119	0.4067
full	so	5	213	57	0.3885	0.3742
full	so	10	192	57	0.403	0.3863
full	so	20	185	50	0.4113	0.404
full	so	40	184	50	0.4119	0.4067
full	so	80	184	50	0.4119	0.4067
full	bs	5	213	406	0.3159	0.3756
full	bs	10	192	406	0.3304	0.3858
full	bs	20	185	406	0.3381	0.3946
full	bs	40	184	406	0.3389	0.3973
full	bs	80	184	406	0.3389	0.3973
nonoise	bso	5	199	35	0.4297	0.4109
nonoise	bso	10	179	35	0.4508	0.4272
nonoise	bso	20	174	35	0.4557	0.4396
nonoise	bso	40	174	35	0.4557	0.4396
nonoise	bso	80	174	35	0.4557	0.4396
nonoise	so	5	199	35	0.4297	0.4109
nonoise	so	10	179	35	0.4508	0.4272
nonoise	so	20	174	35	0.4557	0.4396
nonoise	so	40	174	35	0.4557	0.4396
nonoise	so	80	174	35	0.4557	0.4396
nonoise	bs	5	199	299	0.3611	0.3559
nonoise	bs	10	179	299	0.3817	0.3699
nonoise	bs	20	174	299	0.387	0.3796
nonoise	bs	40	174	299	0.387	0.3796
nonoise	bs	80	174	299	0.387	0.3796

3.5 Quality of the architecture over time

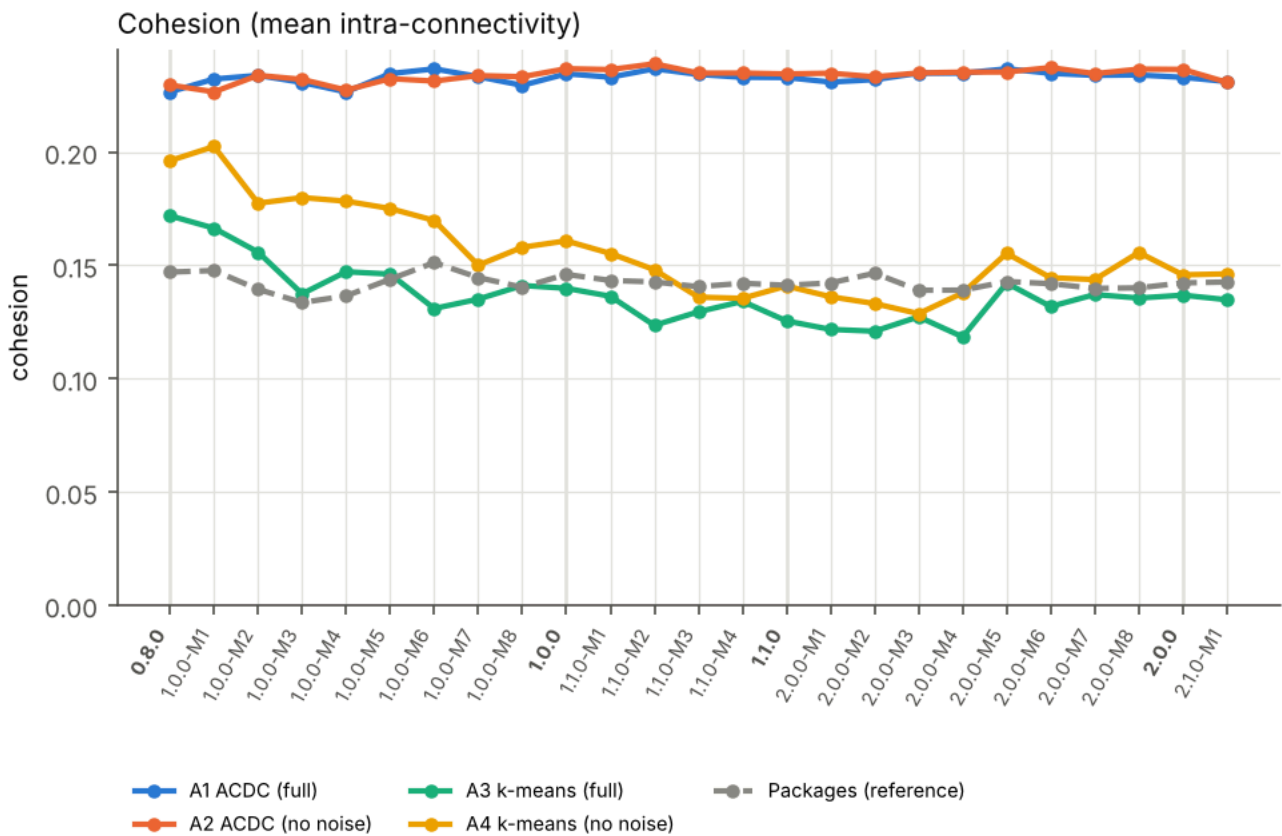


Figure 10 - Cohesion (mean intra-connectivity).

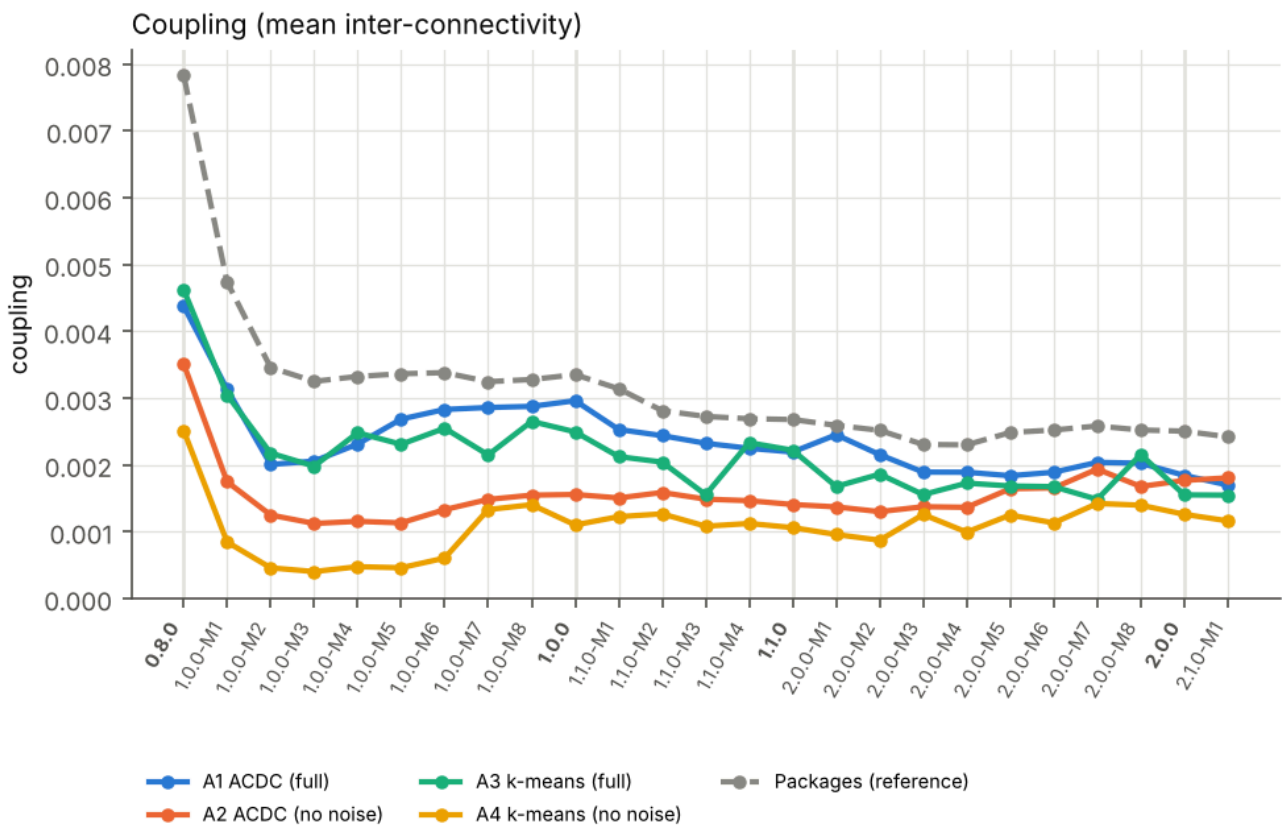


Figure 11 - Coupling (mean inter-connectivity).

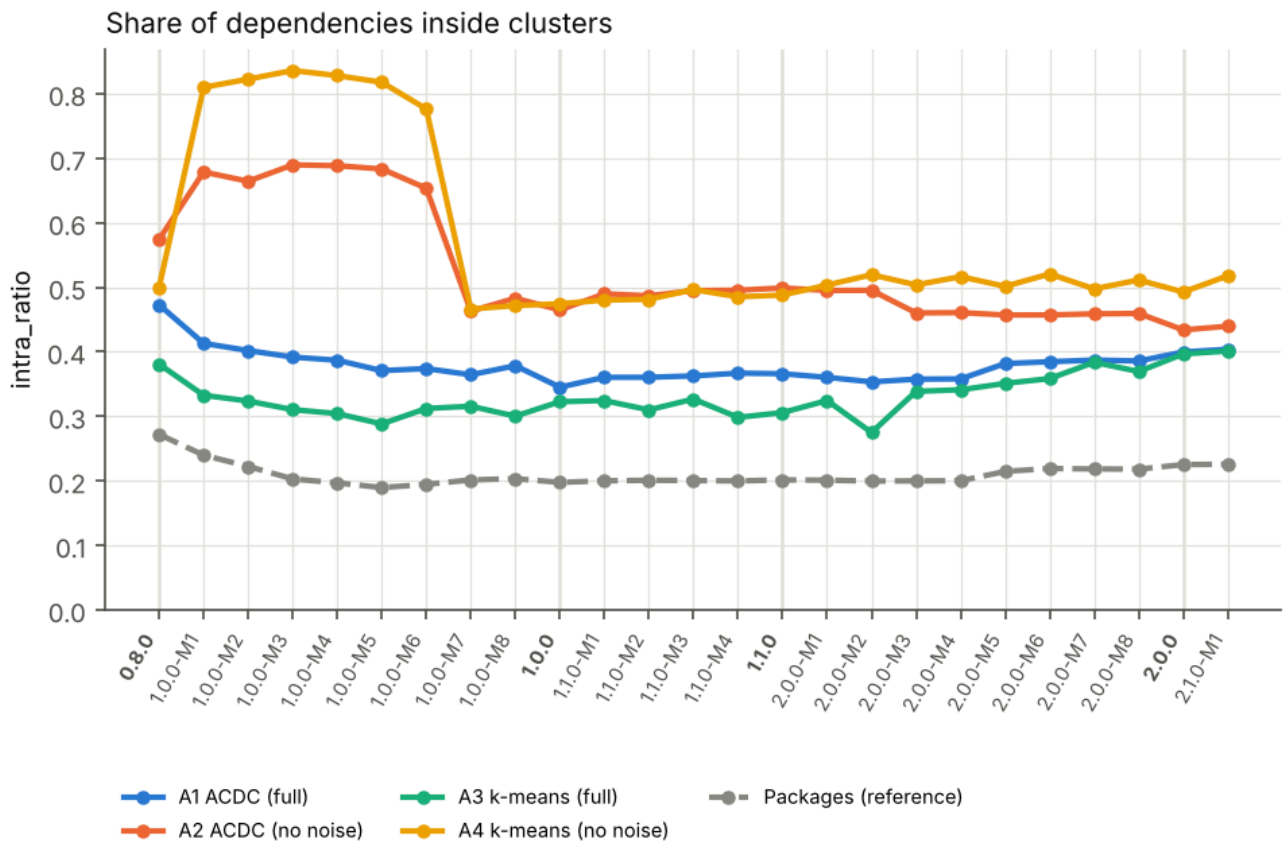


Figure 12 - Share of dependencies inside clusters.

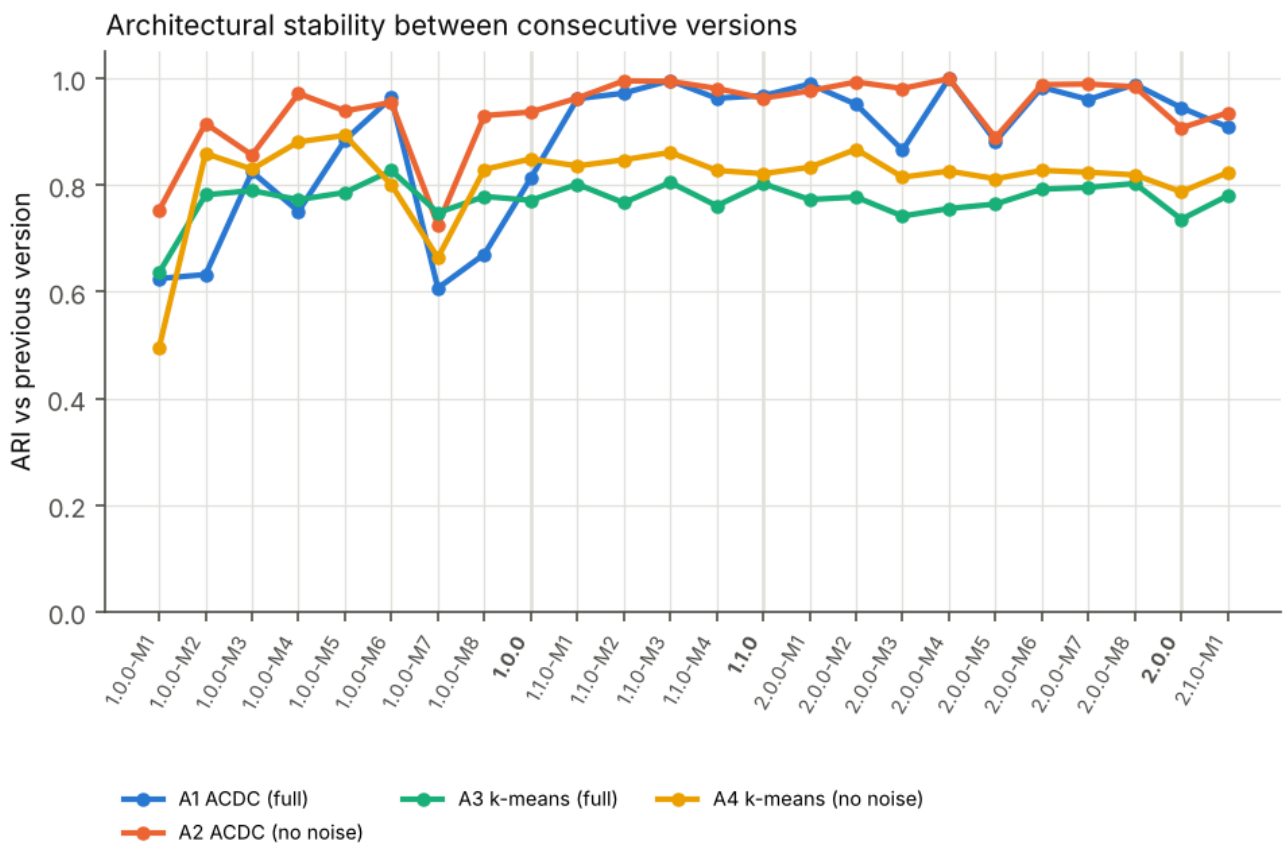


Figure 13 - Stability: ARI between the architectures of consecutive versions.

The evolution of the metrics (Tables 5, 7, 8; Figures 7, 10–13) tells a consistent story in three phases:

1. **0.8.0 → 1.0.0-M6 - erosion during fast growth.** The modularity of the full system decreases: ACDC TurboMQ/k falls from 0.46 to 0.40, the package decomposition from 0.25 to 0.19, and the intra-cluster share of A1 from 0.47 to 0.37. More and more classes depend on a growing set of core abstractions, which is *declining quality (VII)* while new features are added quickly. The architectures are also unstable between releases (A1 ARI 0.62–0.88 for M1–M5): new features are not only added but re-wired.
2. **1.0.0-M7 → 1.0.0 - restructuring.** The module split is the largest architectural change in the history (A1 ARI 0.61 vs 1.0.0-M6). After it, the noise-free architectures (A2, A4) lose quality (A2 0.58 → 0.51) because fewer hub classes are removed (3.3). The full-graph architectures stay at their level.
3. **1.1.0-M1 → 2.1.0-M1 - stable.** ACDC TurboMQ/k stays at 0.40–0.42 while its intra-cluster share rises from 0.36 to 0.40; k-means rises from 0.53 to 0.57, and coupling falls (A1 0.0030 in 1.0.0 → 0.0017). ACDC architectures are **very stable** between releases (ARI 0.94–1.00 for A2 between 1.0.0 and 2.0.0-M4). The visible dips are 2.0.0-M3 (MCP annotations) and 2.0.0-M5 (provider removals). The exception is A2, which falls from 0.55 to 0.46 in 2.0.0/2.1.0-M1: JNode’s noise set drifts towards the MCP classes and leaves real hubs such as Document in the graph (3.3). Despite +33 % classes since 1.0.0, the quality of the full system did not decline. The clean-up releases kept the architecture in shape.

Overall, Lehman’s *declining quality* law holds only for the early, fast-growing phase. After 1.0 the project counteracts it with explicit restructuring, enforced module rules (design doc 02) and removal of adapters, so quality stays stable.

Stability of the methods. ACDC is clearly more stable than k-means (mean ARI 0.88/0.94 for A1/A2 vs 0.77/0.81 for A3/A4). ACDC’s patterns are deterministic and local, so a release changes only the clusters it touches. k-means re-partitions the whole space whenever k or the embedding changes. For evolution studies ACDC therefore gives more interpretable diffs.

3.6 Distance to the reference architecture (MoJoFM)

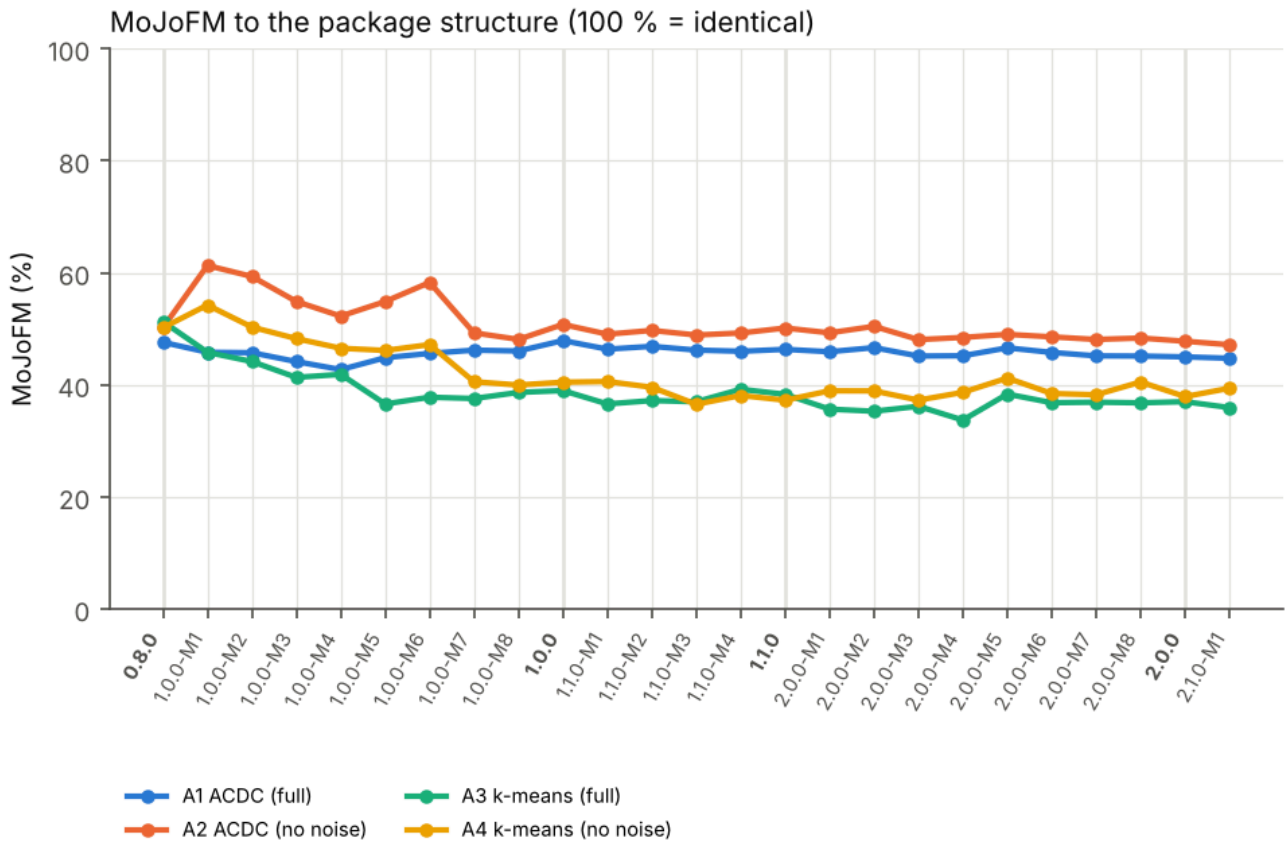


Figure 14 - MoJoFM of A1-A4 to the package structure.

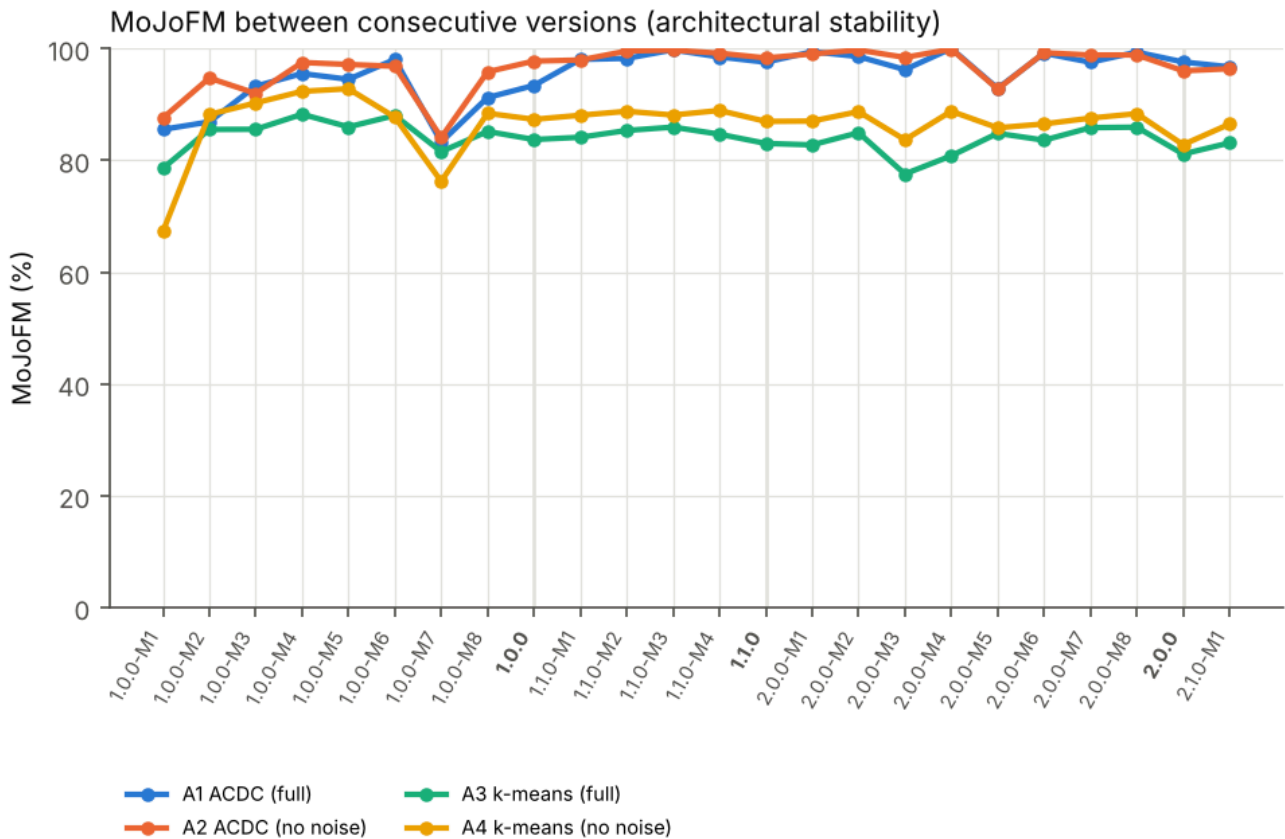


Figure 15 - MoJoFM between consecutive versions.

Spring AI has no expert-validated architecture, so, following the lecture’s methodology, we measure how far each recovered architecture is from a reference architecture: the developers’ package structure (all versions) and the AI architecture (latest version).

- **To the packages** (Table 13): ACDC is closer than k-means (mean MoJoFM 45.7 % for A1 vs 38.6 % for A3), and **removing the noise classes brings both closer** (A2 50.9 %, A4 41.8 %). This is the effect the lecture reports for the noise-removal technique (slide 49). The gain is largest in 1.0.0-M1...M6 (A2 up to 61 %), where JNode’s noise set contained the real hubs. Over time k-means drifts away from the packages ($\approx 50\% \rightarrow 36\text{--}40\%$) while ACDC stays at 43–48 %. As the system grows, connectivity-based clusters cut across Spring AI’s feature/vendor packages more and more.
- **To the AI architecture** (latest version, Table 14): the package structure is closest (73 %), then k-means (65 %), then ACDC (54–55 %). k-means’ fewer, larger clusters are easier to merge into the AI’s 12 components than ACDC’s 185 small ones. (MoJoFM is asymmetric: in the other direction, turning the 12 AI components into 282 packages needs many moves, and MoJoFM(AI $\rightarrow$ packages) is only 12 %.)
- **Stability** (Table 15): consecutive ACDC architectures are almost the same (mean MoJoFM 95.5 % for A1, 96.5 % for A2; lowest 83 % at 1.0.0-M7, the module split). k-means changes much more (84–87 %). This confirms the ARI result of 3.5.

Table 13 – MoJoFM (%) of the recovered architectures to the package structure.

Version	A1 $\rightarrow$ PKG	A2 $\rightarrow$ PKG	A3 $\rightarrow$ PKG	A4 $\rightarrow$ PKG
0.8.0	47.6	50.2	51.29	50.2
1.0.0-M1	45.81	61.3	45.81	54.24
1.0.0-M2	45.74	59.34	44.16	50.31
1.0.0-M3	44.15	54.88	41.33	48.25
1.0.0-M4	42.72	52.26	41.87	46.53
1.0.0-M5	44.84	54.87	36.59	46.14
1.0.0-M6	45.61	58.27	37.79	47.18
1.0.0-M7	46.19	49.27	37.56	40.55
1.0.0-M8	46.04	48.11	38.65	39.97
1.0.0	47.9	50.78	38.97	40.47
1.1.0-M1	46.37	49.04	36.59	40.59
1.1.0-M2	46.87	49.71	37.19	39.57
1.1.0-M3	46.2	48.89	36.98	36.53
1.1.0-M4	45.99	49.25	39.21	38.1
1.1.0	46.33	50.14	38.31	37.26
2.0.0-M1	45.91	49.28	35.66	38.93
2.0.0-M2	46.64	50.44	35.29	38.92
2.0.0-M3	45.14	48.07	36.17	37.27
2.0.0-M4	45.19	48.46	33.71	38.66
2.0.0-M5	46.6	49.01	38.35	41.21
2.0.0-M6	45.78	48.56	36.81	38.45
2.0.0-M7	45.17	48.1	36.9	38.23
2.0.0-M8	45.14	48.35	36.79	40.51
2.0.0	44.99	47.78	36.99	37.9
2.1.0-M1	44.74	47.16	35.94	39.4

Table 14 – MoJoFM (%) to the AI architecture (latest version).

MoJoFM (%) to the AI architecture	A1	A2	A3	A4	Packages
2.1.0-M1	53.69	55.13	65.44	65.4	73.11

Table 15 – MoJoFM (%) between consecutive versions.

Version (vs previous)	A1	A2	A3	A4
1.0.0-M1	85.53	87.58	78.57	67.3
1.0.0-M2	86.88	94.72	85.52	88.2
1.0.0-M3	93.27	91.97	85.55	90.27
1.0.0-M4	95.52	97.5	88.24	92.34
1.0.0-M5	94.49	97.15	85.98	92.8
1.0.0-M6	98.05	96.82	88.02	87.72
1.0.0-M7	83.27	84.08	81.56	76.25
1.0.0-M8	91.23	95.83	85.16	88.41
1.0.0	93.39	97.75	83.72	87.33
1.1.0-M1	98.05	97.94	84.12	88.06
1.1.0-M2	98.23	99.55	85.34	88.77
1.1.0-M3	99.76	99.71	85.94	88.1
1.1.0-M4	98.49	99.16	84.67	88.92
1.1.0	97.55	98.32	83.02	86.96
2.0.0-M1	99.43	99.03	82.76	87.02
2.0.0-M2	98.56	99.73	84.94	88.71
2.0.0-M3	96.22	98.41	77.56	83.67
2.0.0-M4	100.0	99.89	80.81	88.78
2.0.0-M5	92.86	92.74	84.83	85.82
2.0.0-M6	99.16	99.24	83.6	86.51
2.0.0-M7	97.54	98.84	85.84	87.55
2.0.0-M8	99.36	98.85	85.93	88.35
2.0.0	97.59	96.01	81.12	82.8
2.1.0-M1	96.72	96.38	83.12	86.48

3.7 Omnipresent classes: JNode vs Bunch

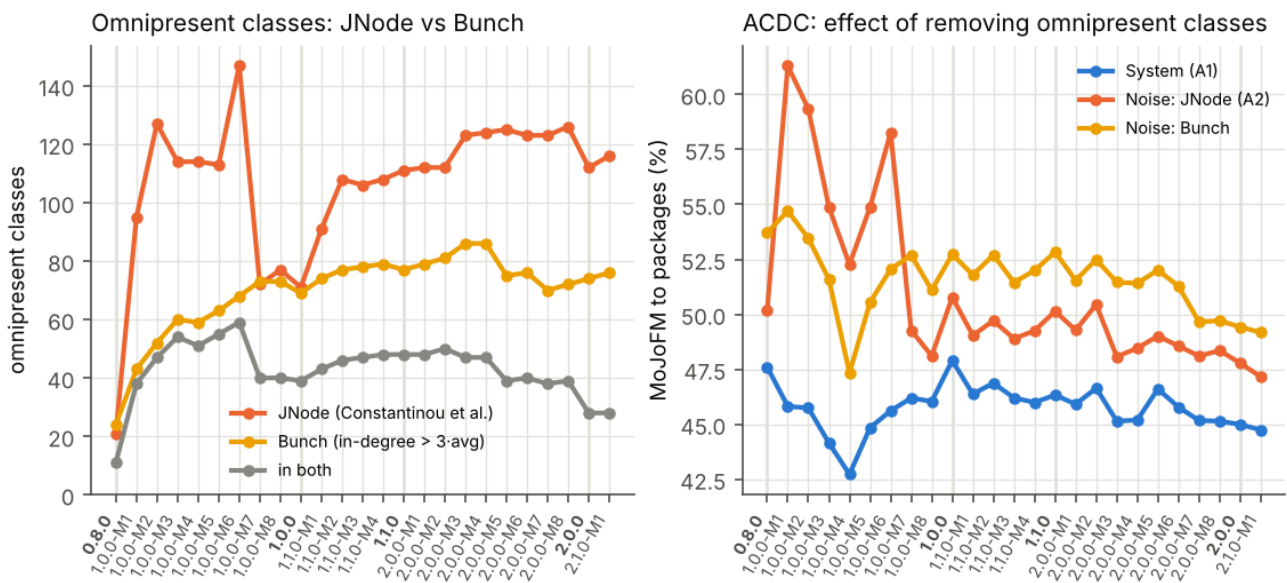


Figure 16 – Omnipresent classes found by JNode and by the Bunch rule (left); MoJoFM of ACDC to the packages for the full system, without JNode noise and without Bunch omnipresent classes (right).

The Bunch rule (in-degree $> 3 \times$ average, Lecture 6–7) flags **24–86 classes** per version, JNode 21–147. The two sets **overlap only partly** (Jaccard 0.17–0.46), and the overlap shrinks over time (0.17 in 2.0.0 and 2.1.0-M1). Bunch flags the classes with the highest fan-in by definition (Document, Filter, EmbeddingModel, ...). JNode’s SIG also flags less used classes that lie on many paths (e.g. the MCP transports, 3.3).

Removing the omnipresent classes before ACDC (Table 16) gives the lecture’s picture: both variants are closer to the package structure than the full system (MoJoFM 47–55 % and 47–61 % vs 43–48 %). **JNode helps more in 1.0.0-M1...M6** (its noise set then contained the real hubs), and **Bunch helps more from 1.0.0-M7 on** (49–53 % vs 47–50 %), when JNode’s set drifts away from the most-used classes. The course MQ is practically the same for all three variants (0.22–0.24). The choice of omnipresent classes changes *which* classes end up together, not how dense the clusters are.

Table 16 – JNode vs Bunch omnipresent classes and their effect on ACDC.

Version	Bunch (>3·avg)	JNode	Both	Jaccard	MQ System	MQ JNode	MQ Bunch	MoJoFM System	MoJoFM JNode	MoJoFM Bunch
0.8.0	24	21	11	0.324	0.222	0.226	0.226	47.6	50.2	53.72
1.0.0-M1	43	95	38	0.38	0.229	0.225	0.236	45.81	61.3	54.7
1.0.0-M2	52	127	47	0.356	0.232	0.233	0.238	45.74	59.34	53.46
1.0.0-M3	60	114	54	0.45	0.229	0.231	0.234	44.15	54.88	51.59
1.0.0-M4	59	114	51	0.418	0.224	0.226	0.226	42.72	52.26	47.33
1.0.0-M5	63	113	55	0.455	0.232	0.231	0.231	44.84	54.87	50.54
1.0.0-M6	68	147	59	0.378	0.234	0.230	0.230	45.61	58.27	52.05
1.0.0-M7	73	72	40	0.381	0.230	0.232	0.231	46.19	49.27	52.69
1.0.0-M8	73	77	40	0.364	0.227	0.232	0.230	46.04	48.11	51.1
1.0.0	69	71	39	0.386	0.232	0.235	0.231	47.9	50.78	52.73
1.1.0-M1	74	91	43	0.352	0.230	0.235	0.233	46.37	49.04	51.79
1.1.0-M2	77	108	46	0.331	0.234	0.237	0.235	46.87	49.71	52.7
1.1.0-M3	78	106	47	0.343	0.232	0.233	0.231	46.2	48.89	51.45
1.1.0-M4	79	108	48	0.345	0.231	0.233	0.230	45.99	49.25	51.99
1.1.0	77	111	48	0.343	0.231	0.233	0.230	46.33	50.14	52.83
2.0.0-M1	79	112	48	0.336	0.228	0.233	0.230	45.91	49.28	51.55
2.0.0-M2	81	112	50	0.35	0.230	0.232	0.228	46.64	50.44	52.49
2.0.0-M3	86	123	47	0.29	0.233	0.233	0.236	45.14	48.07	51.46
2.0.0-M4	86	124	47	0.288	0.233	0.234	0.236	45.19	48.46	51.41
2.0.0-M5	75	125	39	0.242	0.235	0.234	0.234	46.6	49.01	52.01
2.0.0-M6	76	123	40	0.252	0.233	0.236	0.233	45.78	48.56	51.28
2.0.0-M7	70	123	38	0.245	0.232	0.233	0.233	45.17	48.1	49.65
2.0.0-M8	72	126	39	0.245	0.232	0.235	0.233	45.14	48.35	49.71
2.0.0	74	112	28	0.177	0.231	0.235	0.234	44.99	47.78	49.42
2.1.0-M1	76	116	28	0.171	0.229	0.229	0.232	44.74	47.16	49.18

3.8 Architectural smells and their evolution

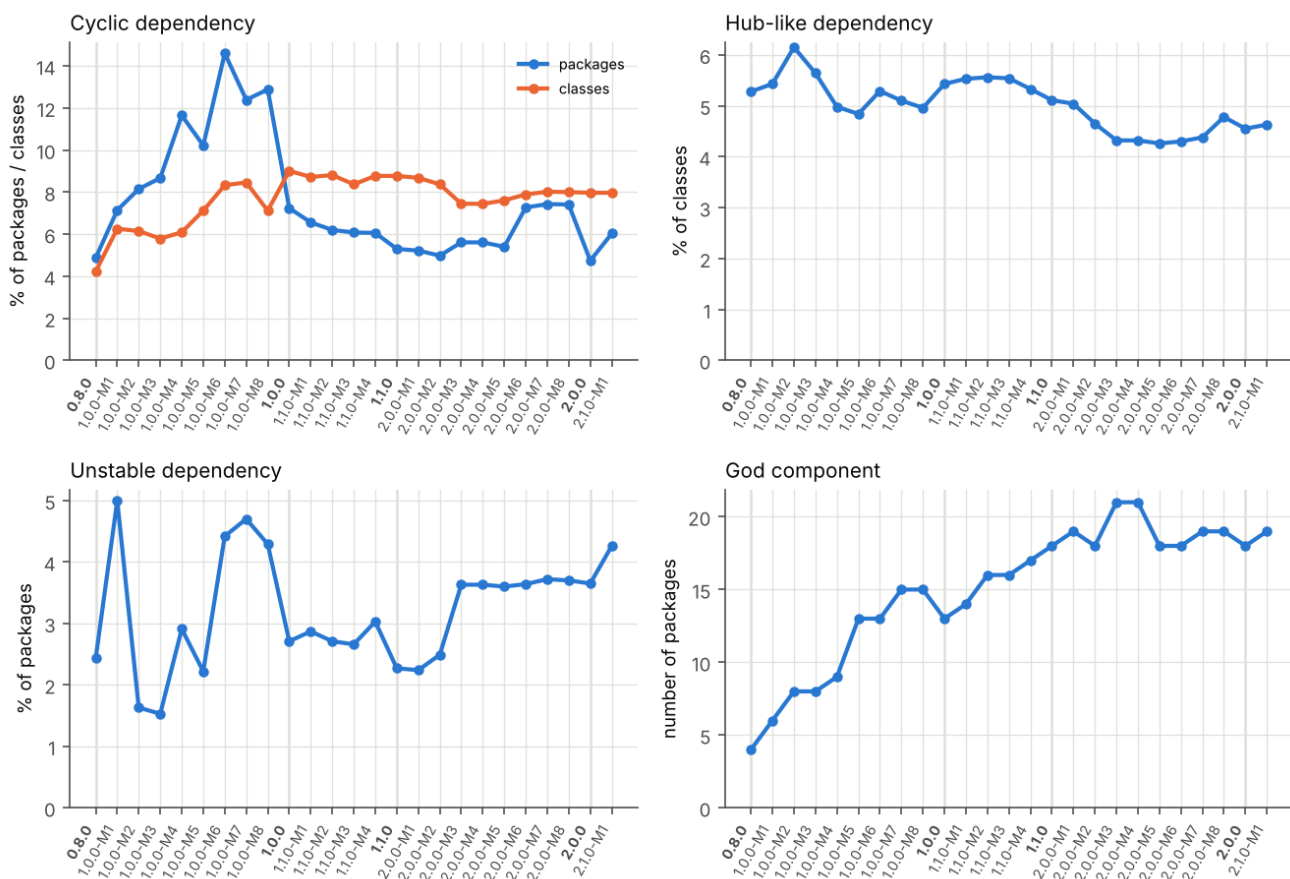


Figure 17 - Evolution of the architectural smells (normalised by #packages or #classes).

Following Lecture 5 (Sas et al.), we track four smells over the 25 versions (Table 17, Figure 17):

- **Cyclic dependency** shows the clearest trend. The share of packages in a dependency cycle rises from 4.9 % (0.8.0) to **14.6 % in 1.0.0-M6**, stays high in M7/M8, and **drops to 7.2 % in 1.0.0** and $\approx 5\text{--}7\%$ afterwards. The release that cleaned up the RAG and chat-memory APIs for the GA removed many package cycles, the same anti-regressive work seen in 3.2. Class-level cycles grow from 4 % to 9 % (1.0.0) and then stay at 7-9 %.
- **Hub-like dependency** stays at 4-6 % of the classes and decreases slightly after 1.1 (5.5 % $\rightarrow$ 4.6 %). No class is getting worse as a hub.
- **Unstable dependency** affects only 1.5-4.7 % of the packages. Spring AI's packages mostly depend on more stable ones (the model API, document, util), as a ports-and-adapters design should.
- **God component.** Spring AI's packages are small (mean ≈ 4 classes), so the mean + 2σ threshold is only 9-10 classes and 4 $\rightarrow$ 19-21 packages exceed it (chat.client, chat.prompt, model, openai, ...). None of them is a "god" package in absolute terms. The growth reflects more feature packages of normal size, not a degrading package.

Table 17 - Architectural smells per version.

Version	Cyclic pkgs %	Cyclic classes %	Hub-like %	Unstable dep. %	God comp.	God components
0.8.0	4.88	4.23	5.28	2.44	4	chat.prompt huggingface.model image vectorstore
1.0.0-M1	7.14	6.26	5.43	5.0	6	chat.prompt huggingface.model image model openai vectorstore
1.0.0-M2	8.15	6.15	6.15	1.63	8	chat.prompt embedding huggingface.model model moderation observation.conventions ...
1.0.0-M3	8.67	5.79	5.64	1.53	8	chat.prompt embedding huggingface.model model moderation observation.conventions ...
1.0.0-M4	11.65	6.09	4.98	2.91	9	chat.prompt embedding huggingface.model model model.function moderation ...
1.0.0-M5	10.22	7.12	4.84	2.22	13	azure.openai chat.client.advisor chat.metadata chat.observation chat.prompt embedding ...
1.0.0-M6	14.6	8.33	5.29	4.42	13	azure.openai chat.client.advisor chat.metadata chat.observation chat.prompt embedding ...
1.0.0-M7	12.39	8.46	5.1	4.7	15	azure.openai chat.metadata chat.observation chat.prompt embedding huggingface.model ...
1.0.0-M8	12.88	7.12	4.96	4.29	15	azure.openai chat.client.advisor chat.client.advisor.api chat.metadata chat.observation chat.prompt ...
1.0.0	7.24	9.01	5.43	2.71	13	azure.openai chat.client chat.metadata chat.prompt embedding huggingface.model ...
1.1.0-M1	6.56	8.72	5.53	2.87	14	azure.openai chat.client chat.metadata chat.prompt embedding huggingface.model ...
1.1.0-M2	6.2	8.81	5.56	2.71	16	azure.openai chat.client chat.metadata

Version	Cyclic pkgs %	Cyclic classes %	Hub-like %	Unstable dep. %	God comp.	God components
						chat.prompt embedding huggingface.model ...
1.1.0-M3	6.08	8.37	5.54	2.66	16	azure.openai chat.client chat.metadata chat.prompt embedding huggingface.model ...
1.1.0-M4	6.06	8.77	5.33	3.03	17	azure.openai chat.client chat.client.advisor chat.metadata chat.prompt embedding ...
1.1.0	5.3	8.77	5.11	2.27	18	azure.openai chat.client chat.client.advisor chat.metadata chat.prompt converter ...
2.0.0-M1	5.22	8.68	5.04	2.24	19	azure.openai chat.client chat.client.advisor chat.metadata chat.prompt converter ...
2.0.0-M2	4.98	8.37	4.65	2.49	18	azure.openai chat.client chat.client.advisor chat.metadata chat.prompt converter ...
2.0.0-M3	5.61	7.45	4.32	3.63	21	anthropic azure.openai chat.client chat.client.advisor chat.metadata chat.prompt ...
2.0.0-M4	5.61	7.44	4.32	3.63	21	anthropic azure.openai chat.client chat.client.advisor chat.metadata chat.prompt ...
2.0.0-M5	5.4	7.61	4.26	3.6	18	anthropic chat.client chat.client.advisor chat.metadata chat.prompt converter ...
2.0.0-M6	7.27	7.89	4.3	3.64	18	anthropic chat.client chat.client.advisor chat.metadata chat.prompt converter ...
2.0.0-M7	7.43	8.02	4.38	3.72	19	anthropic chat.client chat.client.advisor chat.client.advisor.api chat.metadata chat.prompt ...

Version	Cyclic pkgs %	Cyclic classes %	Hub-like %	Unstable dep. %	God comp.	God components
2.0.0-M8	7.41	8.0	4.78	3.7	19	anthropic chat.client chat.client.advisor chat.client.advisor.api chat.metadata chat.prompt ...
2.0.0	4.74	7.97	4.55	3.65	18	anthropic chat.client chat.client.advisor chat.client.advisor.api chat.metadata chat.prompt ...
2.1.0-M1	6.03	7.97	4.63	4.26	19	anthropic chat.client chat.client.advisor chat.client.advisor.api chat.metadata chat.prompt ...

3.9 Lehman's laws: dependency changes and work rate

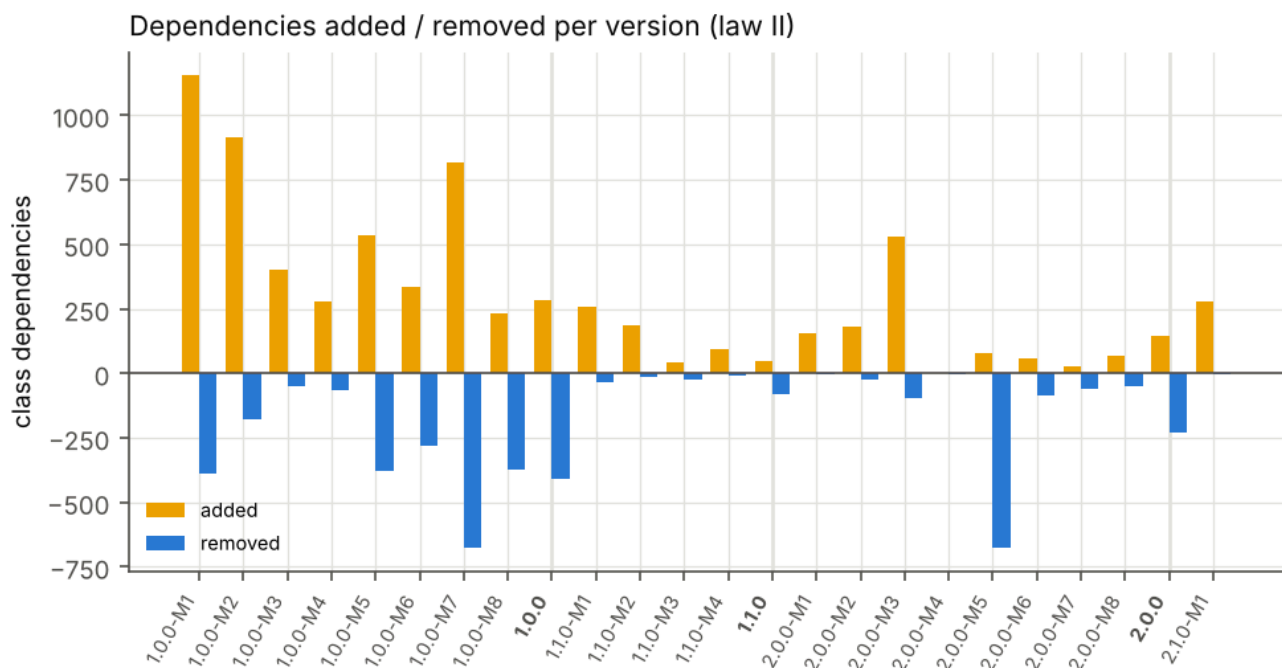


Figure 18 – Dependencies added and removed per version.

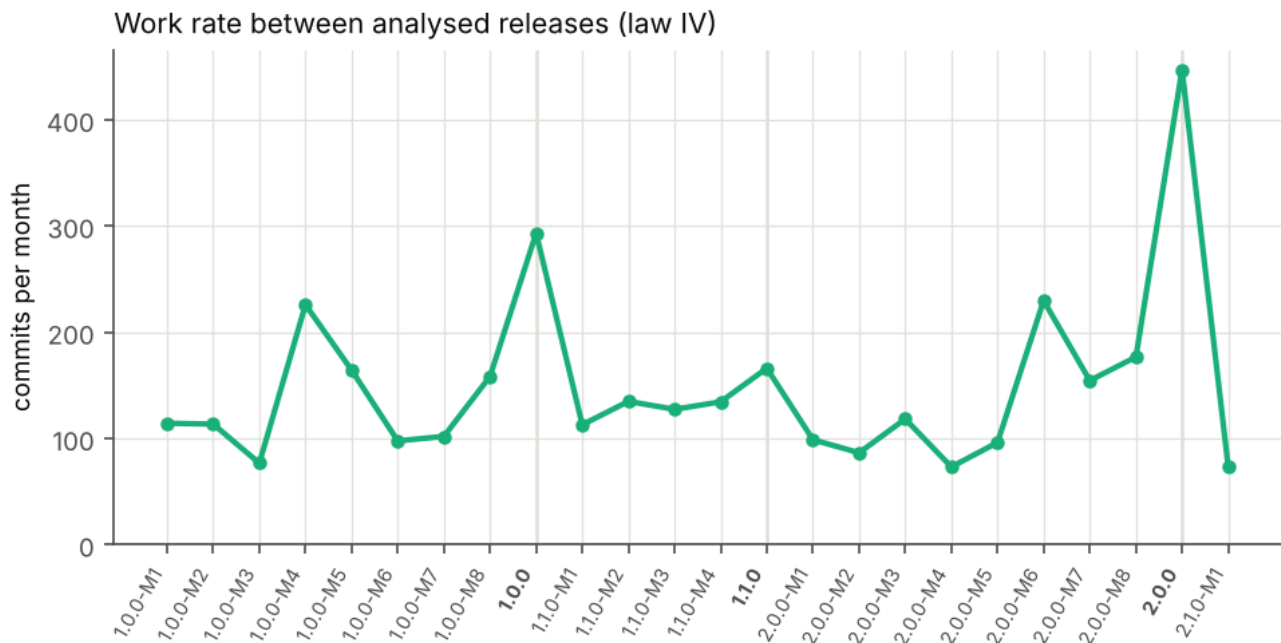


Figure 19 – Commits per month between the analysed releases.

- **Law II - increasing complexity.** As in the lecture's Eclipse example, Figure 18 separates added from removed dependencies. Growth phases add hundreds of dependencies (1,154 in 1.0.0-M1, 916 in M2, 532 in 2.0.0-M3), and **large removals mark the restructuring releases**: 1.0.0-M7 (−678, the module split), 1.0.0-M8 and 1.0.0 (−375, −411, the GA clean-up) and 2.0.0-M5 (−674, removed providers). These are exactly the points where “work is done to reduce complexity”. After each of them the dependency density stays lower (3.2).
- **Law IV - conservation of organisational stability.** The work rate between releases varies from 73 to 447 commits per month (median 123) without a long-term trend; the peaks are the weeks before a GA (1.0.0: 293, 2.0.0: 447). This is consistent with an invariant average work rate.
- **Law VIII - feedback system.** Spring AI's process has the multi-level feedback loops the law describes: public milestones every 1-2 months with user feedback, parallel maintenance lines (1.0.x, 1.1.x, 2.0.x), and design records that turn feedback into enforced rules (design doc 02).
- Spring AI is an **E-type** system: it must keep adapting to new AI providers and protocols (MCP), so all laws apply.

Table 18 – Dependencies added and removed between versions.

Version	Dependencies	Added	Removed
1.0.0-M1	1593	1154	388
1.0.0-M2	2329	916	180
1.0.0-M3	2680	400	49
1.0.0-M4	2895	281	66
1.0.0-M5	3053	534	376
1.0.0-M6	3104	334	283
1.0.0-M7	3241	815	678
1.0.0-M8	3097	231	375
1.0.0	2969	283	411
1.1.0-M1	3194	258	33
1.1.0-M2	3365	185	14
1.1.0-M3	3387	45	23
1.1.0-M4	3472	94	9
1.1.0	3440	50	82
2.0.0-M1	3590	155	5
2.0.0-M2	3746	181	25
2.0.0-M3	4181	532	97
2.0.0-M4	4181	4	4
2.0.0-M5	3583	76	674
2.0.0-M6	3554	58	87
2.0.0-M7	3520	25	59
2.0.0-M8	3538	67	49
2.0.0	3451	143	230
2.1.0-M1	3725	277	3

4. Architecture recovery with AI (latest version, 2.1.0-M1)

Tool and protocol. We used **Claude (Anthropic)** through Claude Code. We wrote four prompts with increasing context (all prompts, inputs and verbatim responses are in ai/):

Prompt	Information given
P1	Only the name/version and the GitHub URL
P2	README of 2.1.0-M1 + list of the 116 Maven modules with their number of classes (ai/inputs/modules.md)
P3	P2 + all 283 packages with class counts and sample classes (packages.md) + the package dependency graph from DependencyExtractor (1,047 weighted edges, package_deps.md); the answer had to map every package to exactly one component as ordered regex rules in JSON
P4	Follow-up: the metrics of the P3 architecture compared with ACDC/k-means, asking for an explanation and possible improvements

Limitation: the same assistant also helped to build the pipeline and had therefore seen the analysis data. The P1 answer is thus not a fully “cold” answer. Re-running the prompts in

ChatGPT or another tool would give an independent comparison.

What the AI produced.

- **P1 (no context):** a plausible 14-component layered architecture (commons, chat/other model APIs, ChatClient & advisors, structured output, tools, memory, RAG/ETL, vector store API and implementations, providers, MCP, observability, Boot auto-configuration) with the right styles (*Ports & Adapters*, plug-in auto-configuration, *Chain of Responsibility* for advisors). But it is generic and partly **outdated**: it lists Azure OpenAI, Vertex AI Gemini, Hugging Face and spring-ai-core, all of which no longer exist in 2.1.0-M1 (removed in 1.0.0-M7, 2.0.0-M3 and 2.0.0-M5). It says it is unsure about exactly these points.
- **P2 (README + modules):** 13 components in 5 layers (Foundation → Core abstractions → Application services → Adapters → Spring Boot integration) with an exact module-to-component mapping. It also identified real smells: spring-ai-model as a large “kernel” (346 class files), MCP annotations as a second large subsystem (224), and 57 auto-configuration modules.
- **P3 (packages + dependency graph):** 12 components with an explicit package mapping (ai/ai_architecture_P3.json), a layered diagram derived from the measured dependencies, the strongest component dependencies (Model providers → Model API: 516; Vector stores → Vector store API: 190), three small cycles (Model API ⇌ Commons ⇌ Tool calling) and the hub packages (document, embedding, vectorstore, model, observation conventions).

Table 10 - Components of the AI architecture (P3) - classes with at least one dependency.

Component	Classes
Boot auto-configuration	214
MCP	163
Model API (chat, embedding, image, audio, moderation)	157
Model providers	147
Vector store implementations	62
Tool calling & tool search	58
ChatClient & Advisors	55
Commons & infrastructure	45
RAG & ETL	39
Vector store API & filter language	29
Chat memory repositories	24
Dev services (Docker Compose / Testcontainers)	23

Table 11 - AI (P3) architecture vs algorithmic architectures (2.1.0-M1).

Architecture (2.1.0-M1)	k	Largest	Cohesion	Coupling	MQ (course)	TurboMQ	TurboMQ/k	Intra deps
AI (P3), full	12	214	0.02836	0.002618	0.02574	5.48	0.4567	0.4722
AI (P3), no noise	12	174	0.02801	0.00229	0.02572	6.295	0.5246	0.5236
A1 ACDC	185	50	0.23107	0.001699	0.22938	76.096	0.4113	0.404
A2 ACDC-noise	174	35	0.23074	0.001807	0.22893	79.295	0.4557	0.4396
A3 k-means	84	32	0.13482	0.001548	0.13327	47.6	0.5667	0.4011
A4 k-means-noise	76	28	0.14621	0.001163	0.14505	49.218	0.6476	0.5178
Packages	282	19	0.14263	0.002421	0.14021	56.873	0.2017	0.2258

Table 12 - Agreement (ARI) of the AI architecture with the other architectures.

	A1	A2	A3	A4	Packages
AI (full)	0.0529	0.0626	0.0888	0.109	0.0688
AI (no noise)	0.0648	0.0626	0.1117	0.109	0.0791

5. Reflection

5.1 Comparing the architectures of the latest version

- **Granularity.** The AI architecture has **12 components**, ACDC 185 clusters, k-means 84, the package structure 282 packages. Only the AI architecture is at the level of abstraction an architect would draw on a whiteboard. The algorithmic architectures are closer to “modules of collaborating classes” and would need a second, hierarchical step to be read as components.
- **Quality metrics - the verdict depends on the measure.** By the **course MQ**, ACDC is clearly best (0.229), k-means (0.133/0.145) and the packages (0.140) follow, and the AI architecture is the *worst* (0.026). MQ’s cohesion term divides by N_i^2 , so 12 components of up to 214 classes can never be dense. On the **k-independent measures** the picture reverses: with 15× fewer components the AI architecture keeps **a higher share of dependencies inside components than ACDC (47.2 % vs 40.4 %)**, and its TurboMQ/k is also higher (0.457 vs 0.411). Only k-means is better (0.567; without noise 0.648). The course MQ is therefore suitable for comparing architectures with a similar number of clusters (A1–A4), not for comparing a 12-component view with 185 clusters.
- **Agreement.** The AI architecture agrees very little with all algorithmic architectures ($ARI \approx 0.05$ – 0.11) and even with the packages (0.07; ARI is low when the numbers of groups differ so much). ACDC and k-means agree with each other only moderately (ARI 0.29 for A1 vs A3, 0.32 for A2 vs A4), and each agrees with the packages at $ARI \approx 0.24$ – 0.30 . With the lecture’s MoJoFM and the AI architecture as reference, the packages are closest (73 %), then k-means (65 %) and ACDC (54–55 %) (3.6).
- **Different principles.** The AI groups by **responsibility and architectural role**. All provider adapters form one component, all vector-store adapters another, all auto-configuration a third. The algorithms group by **connectivity**: each adapter ends up next to the ports it uses most (an OpenAI cluster with its options, API client and auto-configuration). The AI’s decomposition makes the *Ports & Adapters* style visible: the strongest inter-component dependencies are adapter → port by design. The algorithmic

decompositions show *vertical slices* (feature or vendor) that are easier to change in isolation. Both are valid views: the AI's answers "what are the parts and their roles?", the algorithms answer "what changes together / depends together?".

Our assessment. For understanding and documenting the system, the AI architecture (P3) is the most useful. It is correct with respect to the code (every package is mapped, the dependency directions are measured), it matches the documented intent (design doc 02), and it names its own weak points. ACDC is the best fully automatic method for *tracking evolution*: it has the highest course MQ, it is stable and deterministic (MoJoFM $\approx 96\%$ between consecutive versions), and with noise removal it is the closest to the package structure (MoJoFM 51 %). k-means gives the best modularity numbers but depends on k and on the embedding, and its clusters change most between versions.

5.2 Does the AI tool help? How do information and prompts influence the result?

- **It helps a lot, but only when given facts.** P1 is a fluent but generic answer that mixes the 1.x and 2.x code bases and names removed modules: a typical *hallucination by outdated training data*. Adding the module list (P2) fixed the component inventory. Adding packages and the measured dependency graph (P3) made the answer **verifiable and quantitative**: a complete package mapping, real dependency weights, real cycles.
- **The prompt shapes the result.** Asking for "8-15 components" fixed the granularity. Asking for a mapping in machine-readable form (ordered regex rules) forced completeness and let us evaluate the AI architecture with the same metrics as ACDC/k-means. Feeding the metrics back (P4) produced a reasoned answer: it explained the effect of granularity on the metrics and proposed refinements (split MCP into annotations vs transports, split auto-configuration by target, move observability into its own component) without giving up the port/adaptor boundary.
- **Risks.** The AI does not compute anything on its own; it reasons over what it is given. Without the extracted facts, the result cannot be checked, and even with them, a claim like "small cycles" must be verified (we did, `results/ai_component_deps.csv`). The output also depends on the model version and on the conversation, so it is not reproducible in a strict sense.

5.3 Which method for other systems?

For other systems and many versions we would **combine** the methods: run the deterministic pipeline (DependencyExtractor $\rightarrow$ noise removal $\rightarrow$ ACDC) on every version for *evolution* and *metrics*, and use an AI tool on the **latest** version, *given the extracted facts* (modules, packages, the dependency graph and the ACDC clusters), to produce a named, layered component view and to explain the algorithmic clusters. AI alone is fast but unverifiable and possibly outdated. Algorithms alone are reproducible but produce clusters that still need a human (or an AI) to name and abstract them. Noise removal should always be applied, but the noise set should be sanity-checked (e.g. against fan-in, as in 3.3), because JNode's SIG measure depends on the graph's shape.

6. Threats to validity

- **Binary vs source.** We analyse published jars, so only compiled dependencies are seen (no reflection, Spring wiring by configuration, service loading or annotations processed at run time).
- **System boundary.** The BOM defines the system. Modules not in the BOM (e.g. integration tests, docs) are not analysed. Duplicated split-package classes are kept once.
- **Tool changes.** JNode had to be patched (performance, identical output; precision, Jaccard ≥ 0.88 with the original where it worked), and 0.8.0 needed a threshold fallback. ACDC is a third-party port, not the York original, and needed file-style class names to name its subsystems correctly (2.3).
- **k-means.** The results depend on the feature representation (dependency profiles + SVD) and on the elbow choice of k. The silhouette would pick much finer clusterings (Figure 2).
- **Metrics.** Cohesion, the course MQ and TurboMQ depend strongly on k; we report the course MQ as the main measure for architectures of similar size (A1–A4) and TurboMQ/k, the intra-cluster share and MoJoFM to compare across very different k.
- **Milestones as versions** (see 1.2).

References

- Mancoridis S., Mitchell B. S., Rorres C., Chen Y., Gansner E. R. (1998). *Using Automatic Clustering to Produce High-Level System Organizations of Source Code*. IWPC.
- Mitchell B. S., Mancoridis S. (2006). *On the automatic modularization of software systems using the Bunch tool*. IEEE TSE 32(3).
- Tzerpos V., Holt R. C. (2000). *ACDC: An Algorithm for Comprehension-Driven Clustering*. WCRE.
- Lehman M. M. (1980/1996). *Laws of Software Evolution Revisited*. EWSPT.
- Turski W. M. (1996). *Reference model for smooth growth of software systems*. IEEE TSE 22(8).
- Satopää V. et al. (2011). *Finding a “Kneedle” in a Haystack*. ICDCSW.
- Spring AI reference documentation and design/02-boot-modularity.adoc, v2.1.0-M1.

Appendix A - Full metric tables

Table 5 – Number of clusters (k).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
0.8.0	54	53	36	36	82
1.0.0-M1	85	94	60	58	140
1.0.0-M2	108	121	70	64	184
1.0.0-M3	112	124	64	69	196
1.0.0-M4	110	124	74	69	206
1.0.0-M5	116	131	74	70	225
1.0.0-M6	124	129	74	69	226
1.0.0-M7	142	139	76	74	234
1.0.0-M8	137	129	81	76	233
1.0.0	141	134	81	76	221
1.1.0-M1	143	134	76	76	244
1.1.0-M2	153	142	84	74	258
1.1.0-M3	156	145	84	67	263
1.1.0-M4	156	147	94	74	264
1.1.0	158	149	85	74	264
2.0.0-M1	163	153	85	74	268
2.0.0-M2	169	161	94	76	281
2.0.0-M3	203	185	94	76	303
2.0.0-M4	203	186	84	85	303
2.0.0-M5	184	169	94	84	278
2.0.0-M6	182	169	85	76	275
2.0.0-M7	179	168	85	76	269
2.0.0-M8	179	167	85	84	270
2.0.0	178	167	85	76	274
2.1.0-M1	185	174	84	76	282

Table 7 – TurboMQ/k (normalised TurboMQ).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
0.8.0	0.457	0.533	0.558	0.686	0.254
1.0.0-M1	0.440	0.622	0.529	0.831	0.235
1.0.0-M2	0.421	0.606	0.536	0.855	0.212
1.0.0-M3	0.420	0.629	0.545	0.859	0.204
1.0.0-M4	0.415	0.625	0.514	0.844	0.196
1.0.0-M5	0.397	0.629	0.523	0.835	0.187
1.0.0-M6	0.396	0.582	0.516	0.795	0.196
1.0.0-M7	0.402	0.506	0.511	0.645	0.194
1.0.0-M8	0.405	0.524	0.490	0.660	0.189
1.0.0	0.402	0.523	0.523	0.681	0.197
1.1.0-M1	0.409	0.532	0.533	0.671	0.192
1.1.0-M2	0.412	0.522	0.506	0.667	0.194
1.1.0-M3	0.415	0.536	0.550	0.690	0.192
1.1.0-M4	0.414	0.539	0.506	0.681	0.193
1.1.0	0.419	0.550	0.513	0.675	0.192
2.0.0-M1	0.414	0.549	0.529	0.694	0.197
2.0.0-M2	0.410	0.549	0.485	0.699	0.202
2.0.0-M3	0.404	0.500	0.531	0.659	0.194
2.0.0-M4	0.404	0.499	0.519	0.657	0.194
2.0.0-M5	0.407	0.480	0.539	0.631	0.202
2.0.0-M6	0.409	0.481	0.537	0.656	0.201
2.0.0-M7	0.410	0.478	0.548	0.623	0.197
2.0.0-M8	0.409	0.479	0.539	0.643	0.197
2.0.0	0.413	0.453	0.553	0.619	0.204
2.1.0-M1	0.411	0.456	0.567	0.648	0.202

Table 7b – TurboMQ.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
0.8.0	24.7	28.3	20.1	24.7	20.8
1.0.0-M1	37.4	58.5	31.7	48.2	32.9
1.0.0-M2	45.5	73.4	37.6	54.7	39.0
1.0.0-M3	47.0	78.0	34.9	59.3	40.0
1.0.0-M4	45.7	77.5	38.1	58.2	40.4
1.0.0-M5	46.0	82.3	38.7	58.5	42.1
1.0.0-M6	49.0	75.1	38.2	54.8	44.3
1.0.0-M7	57.0	70.4	38.9	47.7	45.3
1.0.0-M8	55.4	67.6	39.7	50.2	44.1
1.0.0	56.7	70.1	42.4	51.8	43.5
1.1.0-M1	58.5	71.2	40.5	51.0	46.8
1.1.0-M2	63.0	74.1	42.5	49.3	50.0
1.1.0-M3	64.7	77.7	46.2	46.2	50.5
1.1.0-M4	64.5	79.2	47.6	50.4	51.1
1.1.0	66.3	82.0	43.6	50.0	50.7
2.0.0-M1	67.5	84.0	45.0	51.4	52.8
2.0.0-M2	69.3	88.3	45.6	53.1	56.9
2.0.0-M3	81.9	92.4	49.9	50.1	58.8
2.0.0-M4	81.9	92.8	43.6	55.9	58.8
2.0.0-M5	74.9	81.1	50.7	53.0	56.1
2.0.0-M6	74.4	81.3	45.6	49.9	55.2
2.0.0-M7	73.3	80.2	46.5	47.4	52.9
2.0.0-M8	73.2	80.0	45.8	54.0	53.1
2.0.0	73.5	75.6	47.0	47.1	55.8
2.1.0-M1	76.1	79.3	47.6	49.2	56.9

Table 7c – MQ (course definition).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k- means–noise	Packages
0.8.0	0.2221	0.2260	0.1673	0.1936	0.1392
1.0.0-M1	0.2291	0.2248	0.1633	0.2016	0.1429
1.0.0-M2	0.2318	0.2326	0.1536	0.1769	0.1360
1.0.0-M3	0.2286	0.2309	0.1355	0.1795	0.1303
1.0.0-M4	0.2242	0.2261	0.1447	0.1778	0.1332
1.0.0-M5	0.2319	0.2312	0.1438	0.1746	0.1405
1.0.0-M6	0.2338	0.2299	0.1284	0.1692	0.1478
1.0.0-M7	0.2304	0.2322	0.1329	0.1487	0.1414
1.0.0-M8	0.2265	0.2316	0.1384	0.1566	0.1371
1.0.0	0.2315	0.2352	0.1374	0.1597	0.1426
1.1.0-M1	0.2304	0.2349	0.1342	0.1540	0.1402
1.1.0-M2	0.2343	0.2374	0.1215	0.1468	0.1399
1.1.0-M3	0.2319	0.2333	0.1280	0.1349	0.1379
1.1.0-M4	0.2307	0.2334	0.1318	0.1343	0.1394
1.1.0	0.2306	0.2330	0.1232	0.1398	0.1385
2.0.0-M1	0.2284	0.2334	0.1200	0.1352	0.1396
2.0.0-M2	0.2298	0.2319	0.1191	0.1322	0.1440
2.0.0-M3	0.2329	0.2335	0.1257	0.1274	0.1367
2.0.0-M4	0.2328	0.2339	0.1167	0.1368	0.1367
2.0.0-M5	0.2349	0.2336	0.1405	0.1540	0.1404
2.0.0-M6	0.2328	0.2356	0.1301	0.1434	0.1394
2.0.0-M7	0.2318	0.2326	0.1356	0.1423	0.1373
2.0.0-M8	0.2319	0.2349	0.1334	0.1542	0.1375
2.0.0	0.2311	0.2347	0.1353	0.1446	0.1398
2.1.0-M1	0.2294	0.2289	0.1333	0.1451	0.1402

Table 8a – Cohesion (mean intra-connectivity).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
0.8.0	0.2265	0.2295	0.1719	0.1962	0.1470
1.0.0-M1	0.2322	0.2265	0.1664	0.2024	0.1477
1.0.0-M2	0.2338	0.2338	0.1558	0.1774	0.1394
1.0.0-M3	0.2307	0.2320	0.1374	0.1799	0.1336
1.0.0-M4	0.2265	0.2273	0.1472	0.1783	0.1365
1.0.0-M5	0.2346	0.2323	0.1461	0.1751	0.1439
1.0.0-M6	0.2367	0.2313	0.1310	0.1698	0.1512
1.0.0-M7	0.2332	0.2337	0.1351	0.1500	0.1446
1.0.0-M8	0.2294	0.2332	0.1411	0.1580	0.1403
1.0.0	0.2345	0.2367	0.1399	0.1608	0.1460
1.1.0-M1	0.2330	0.2364	0.1363	0.1552	0.1433
1.1.0-M2	0.2367	0.2389	0.1235	0.1480	0.1427
1.1.0-M3	0.2342	0.2348	0.1295	0.1359	0.1406
1.1.0-M4	0.2329	0.2348	0.1341	0.1354	0.1421
1.1.0	0.2328	0.2344	0.1255	0.1409	0.1412
2.0.0-M1	0.2308	0.2347	0.1217	0.1361	0.1422
2.0.0-M2	0.2319	0.2332	0.1209	0.1331	0.1465
2.0.0-M3	0.2347	0.2349	0.1273	0.1287	0.1390
2.0.0-M4	0.2347	0.2352	0.1184	0.1378	0.1390
2.0.0-M5	0.2367	0.2353	0.1421	0.1553	0.1429
2.0.0-M6	0.2347	0.2372	0.1318	0.1445	0.1419
2.0.0-M7	0.2338	0.2346	0.1371	0.1437	0.1398
2.0.0-M8	0.2340	0.2366	0.1356	0.1557	0.1401
2.0.0	0.2329	0.2364	0.1368	0.1458	0.1422
2.1.0-M1	0.2311	0.2307	0.1348	0.1462	0.1426

Table 8b – Coupling (mean inter-connectivity).

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k-means–noise	Packages
0.8.0	0.00438	0.00352	0.00463	0.00251	0.00784
1.0.0-M1	0.00313	0.00175	0.00305	0.00085	0.00474
1.0.0-M2	0.00201	0.00125	0.00218	0.00046	0.00346
1.0.0-M3	0.00206	0.00112	0.00198	0.00040	0.00325
1.0.0-M4	0.00231	0.00116	0.00248	0.00048	0.00332
1.0.0-M5	0.00269	0.00113	0.00231	0.00046	0.00336
1.0.0-M6	0.00283	0.00133	0.00255	0.00061	0.00338
1.0.0-M7	0.00286	0.00148	0.00215	0.00133	0.00325
1.0.0-M8	0.00288	0.00155	0.00265	0.00140	0.00328
1.0.0	0.00296	0.00156	0.00249	0.00110	0.00335
1.1.0-M1	0.00253	0.00151	0.00213	0.00123	0.00314
1.1.0-M2	0.00244	0.00159	0.00204	0.00127	0.00281
1.1.0-M3	0.00232	0.00149	0.00155	0.00108	0.00273
1.1.0-M4	0.00225	0.00146	0.00233	0.00112	0.00269
1.1.0	0.00219	0.00141	0.00222	0.00106	0.00268
2.0.0-M1	0.00245	0.00137	0.00168	0.00096	0.00259
2.0.0-M2	0.00215	0.00130	0.00186	0.00087	0.00252
2.0.0-M3	0.00189	0.00137	0.00156	0.00126	0.00231
2.0.0-M4	0.00189	0.00137	0.00173	0.00099	0.00230
2.0.0-M5	0.00184	0.00164	0.00169	0.00125	0.00249
2.0.0-M6	0.00189	0.00166	0.00168	0.00113	0.00252
2.0.0-M7	0.00204	0.00194	0.00148	0.00142	0.00258
2.0.0-M8	0.00203	0.00168	0.00216	0.00140	0.00252
2.0.0	0.00184	0.00177	0.00155	0.00126	0.00251
2.1.0-M1	0.00170	0.00181	0.00155	0.00116	0.00242

Table 8c – Share of dependencies inside clusters.

Version	A1 ACDC	A2 ACDC–noise	A3 k-means	A4 k- means–noise	Packages
0.8.0	0.473	0.574	0.381	0.500	0.272
1.0.0-M1	0.413	0.679	0.333	0.810	0.240
1.0.0-M2	0.402	0.664	0.324	0.822	0.222
1.0.0-M3	0.392	0.690	0.310	0.836	0.203
1.0.0-M4	0.387	0.688	0.304	0.828	0.197
1.0.0-M5	0.371	0.683	0.288	0.818	0.190
1.0.0-M6	0.374	0.653	0.312	0.776	0.195
1.0.0-M7	0.364	0.463	0.316	0.466	0.202
1.0.0-M8	0.378	0.483	0.300	0.472	0.203
1.0.0	0.345	0.465	0.323	0.474	0.198
1.1.0-M1	0.360	0.490	0.324	0.480	0.200
1.1.0-M2	0.360	0.487	0.310	0.481	0.201
1.1.0-M3	0.363	0.495	0.327	0.496	0.201
1.1.0-M4	0.367	0.495	0.298	0.485	0.200
1.1.0	0.366	0.499	0.306	0.488	0.202
2.0.0-M1	0.360	0.495	0.325	0.503	0.201
2.0.0-M2	0.353	0.495	0.275	0.519	0.200
2.0.0-M3	0.358	0.460	0.339	0.504	0.200
2.0.0-M4	0.358	0.461	0.341	0.516	0.200
2.0.0-M5	0.382	0.457	0.351	0.501	0.215
2.0.0-M6	0.385	0.457	0.358	0.521	0.219
2.0.0-M7	0.387	0.459	0.384	0.497	0.218
2.0.0-M8	0.386	0.459	0.369	0.511	0.218
2.0.0	0.400	0.434	0.397	0.493	0.225
2.1.0-M1	0.404	0.440	0.401	0.518	0.226

Table 8d – Stability (ARI with the previous version).

Version (vs previous)	A1	A2	A3	A4
1.0.0-M1	0.6248	0.7525	0.6358	0.4951
1.0.0-M2	0.6323	0.9154	0.7817	0.8591
1.0.0-M3	0.8247	0.8562	0.7896	0.8298
1.0.0-M4	0.7508	0.9716	0.7726	0.881
1.0.0-M5	0.8842	0.9389	0.7861	0.8931
1.0.0-M6	0.9636	0.9549	0.8272	0.8005
1.0.0-M7	0.6069	0.7257	0.748	0.6638
1.0.0-M8	0.6705	0.9302	0.7786	0.8293
1.0.0	0.813	0.9369	0.7714	0.8489
1.1.0-M1	0.9617	0.963	0.8017	0.8359
1.1.0-M2	0.9722	0.9953	0.7668	0.8473
1.1.0-M3	0.9959	0.9944	0.805	0.8612
1.1.0-M4	0.9627	0.9811	0.7612	0.8275
1.1.0	0.9676	0.9627	0.8027	0.8213
2.0.0-M1	0.9892	0.9764	0.7729	0.8334
2.0.0-M2	0.9513	0.9932	0.7775	0.8671
2.0.0-M3	0.8653	0.981	0.7419	0.8148
2.0.0-M4	1.0	0.9994	0.7561	0.8262
2.0.0-M5	0.8819	0.8888	0.7644	0.8118
2.0.0-M6	0.9833	0.9883	0.7923	0.8275
2.0.0-M7	0.9603	0.9894	0.7956	0.824
2.0.0-M8	0.9887	0.9842	0.8029	0.8191
2.0.0	0.9449	0.9071	0.7359	0.7874
2.1.0-M1	0.9087	0.9356	0.78	0.8236

Table 9 – Module changes per release.

Version	Modules	Added	Removed
1.0.0-M1	35	18	4
1.0.0-M2	42	7	0
1.0.0-M3	45	3	0
1.0.0-M4	47	3	1
1.0.0-M5	48	1	0
1.0.0-M6	51	3	0
1.0.0-M7	104	57	4
1.0.0-M8	104	4	4
1.0.0	102	9	11
1.1.0-M1	110	10	2
1.1.0-M2	113	3	0
1.1.0-M3	115	3	1
1.1.0-M4	116	1	0
1.1.0	117	2	1
2.0.0-M1	120	3	0
2.0.0-M2	130	10	0
2.0.0-M3	130	2	2
2.0.0-M4	130	0	0
2.0.0-M5	121	0	9
2.0.0-M6	118	0	3
2.0.0-M7	112	0	6
2.0.0-M8	112	1	1
2.0.0	112	4	4
2.1.0-M1	116	4	0

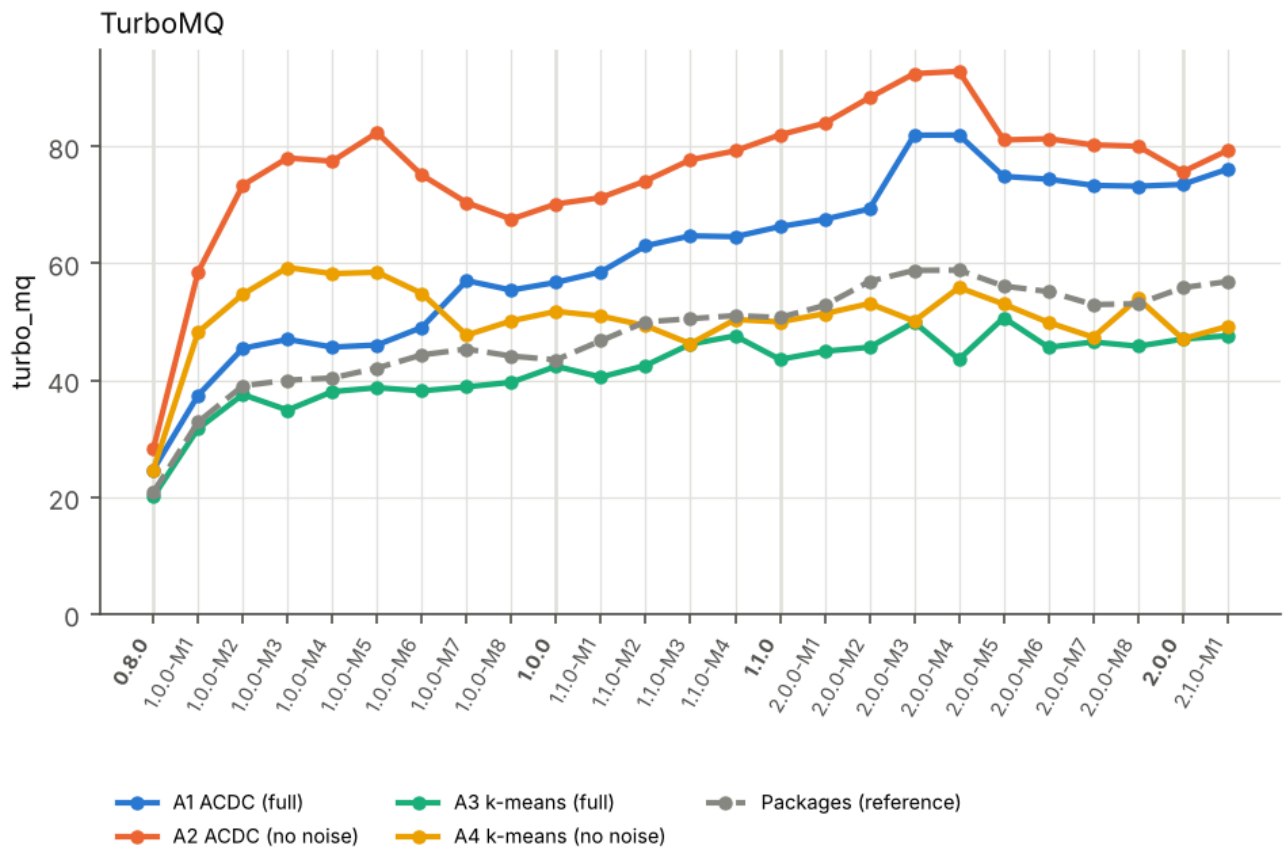


Figure A1 - TurboMQ.

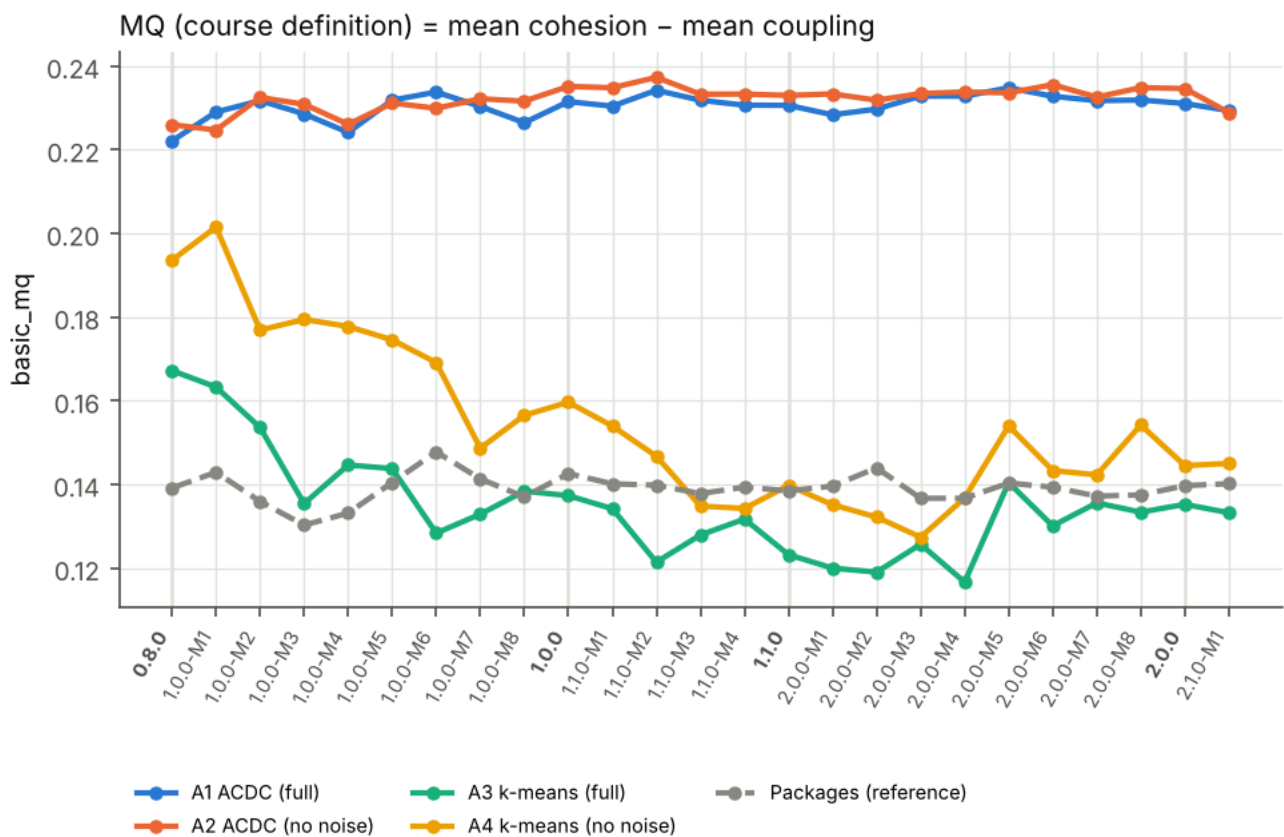


Figure A2 - MQ (course definition).